EHRJEPA: Joint-Embedding Predictive Architectures for Longitudinal Electronic Health Records

A work-in-progress report on five self-supervised objectives, evaluated under matched compute with untrained controls

Abel Yagubyan
University of California, Berkeley
abelyagubyan@berkeley.edu

6 September 2026

Status: work in progress. This document reports the state of an ongoing project. Every number in it comes from a run recorded in the project repository at a named commit. All pretraining is on a single synthetic, claims-derived source (CMS DE-SynPUF) with no laboratory values; no experiment on MIMIC-IV or EHRSHOT has been run. The largest token budget reported (1B) now has three training seeds per cell for `ar` and the hybrid; the 200M budget, and `recon_only` and `jepa_ema` at every budget they were run at, remain one training seed per cell. Nothing here should be read as a statement about what these objectives do at production scale or on clinical data.

Abstract

We implement and compare five self-supervised objectives over longitudinal electronic health record (EHR) event streams in MEDS format: masked-span latent joint-embedding prediction (JEPA), causal next-latent JEPA, window-pooled latent JEPA, next-code autoregression (AR), and a hybrid that carries a dense next-latent term and the next-code term through one trunk. All variants share an event embedding that fuses a learned code embedding with a value quantizer and Fourier time features, a rotary-position encoder, and, for the latent objectives, a stop-gradient target encoder regularized against collapse by a SIGReg-style distributional penalty. We evaluate frozen-encoder probes on seven downstream tasks defined declaratively through ACES, against count-feature logistic-regression and gradient-boosting baselines and against architecture- and pooling-matched untrained (`random_init`) controls, on a fixed seeded 3,000-subject held-out cut with 200 bootstrap resamples.

Across 20 pilot cells trained to a matched 48M-token budget, the five masked-span JEPA variants score mean AUROC 0.6776–0.6901 against a shared untrained control at 0.6821 (gain −0.0046 to +0.0080), while next-code AR scores 0.7205 against its own control at 0.6760 (+0.0445). Adding an auxiliary code-reconstruction term to a masked-span JEPA raises the gain to +0.0177 to +0.0296; removing the latent term entirely and keeping only code reconstruction retains +0.0254. The dense causal hybrid scores 0.7287 (+0.0923), the highest of the 20 cells. Scaling the hybrid and AR to 200M and 1B nominal token slots on a 6-layer, 256-wide encoder, the hybrid improves at every step (0.7287, 0.7328, 0.7363, the last a 3-seed mean) while AR does not improve from 200M to 1B (0.7304, 0.7251, the last a 3-seed mean). At 1B, the hybrid leads AR on six of seven tasks by 0.81 to 2.16 AUROC points, and the per-task min-max range across the three seeds does not overlap between AR and the hybrid on four of those six. Three pre-registered diagnostics measured on the masked-span checkpoints are reported: a ridge map from mask-token time features alone recovers $R^2 = 0.58$ of the layer-normed targets against 0.79 for the trained predictor with context; a linear code-identity probe reads top-1 0.50 off the trained JEPA encoders against 0.60 off an untrained encoder and 0.71 off the AR encoder; and without SIGReg a plain context-mean baseline reaches cosine 0.73 to the targets. We report what the evidence supports and what it does not.

1 Introduction

An electronic health record is a stream of timestamped clinical events — diagnoses, procedures, medication fills, admissions, laboratory results — observed at irregular intervals over years. The dominant self-supervised recipe for such data is next-token autoregression over a serialized code sequence, in the manner of CLMBR [17], ETHOS [14], CEHR-GPT [13] and EHRMamba [8]. That recipe inherits a property of language modelling that is less obviously appropriate here: it spends its entire capacity on predicting the identity of the next discrete symbol, including the large fraction of symbol identity that is administrative, arbitrary, or genuinely unpredictable. Which of several thousand billable codes a clinician selects for the same underlying condition, and which of many interchangeable national drug codes a pharmacy dispenses, are not facts a representation of the patient needs to carry.

What JEPA promises for EHR. Joint-embedding predictive architectures [3, 4] sidestep this by predicting in representation space. A context encoder reads the observed events; a predictor, conditioned on where and when the unobserved events fall, predicts the *representations* a target encoder assigns to them; nothing is reconstructed in input space. The target is produced by a stop-gradient copy of the encoder, so the objective is free to discard detail that is not predictable — exactly the detail an EHR stream is full of. The obvious failure mode, representational collapse, is addressed here by SIGReg [5], which pushes embeddings toward an isotropic Gaussian through random one-dimensional projections rather than through architectural asymmetry or negative pairs. On paper this is a good match for clinical event streams. This paper reports what happened when we measured it.

Contributions. Stated as facts about what exists and what was measured, not as claims about importance:

1. An open implementation of five objective families over a common MEDS [2] data path, a common event embedding, and a common encoder: masked-span latent JEPA, causal next-latent JEPA, window-pooled latent JEPA, next-code AR, and an AR/JEPA hybrid (Section 4).
2. A downstream evaluation harness in which every model sees identical anchors and identical history windows, history is cut by a single strict inequality, tasks are defined declaratively in ACES [19], and every trained cell is scored against an architecture- and pooling-matched untrained control as well as against count-feature baselines (Section 5).
3. Five ablation grids at a matched 48M-token budget — 20 trained cells at seed 0, plus four seed-replication cells — and two token-scaling grids at 200M and 1B nominal token slots, the 1B grid replicated at three seeds each for `ar` and the hybrid (Section 6).
4. The measured result that, at these budgets on this source, masked-span latent prediction alone does not move a frozen-encoder probe past its own untrained control, that an auxiliary discrete code term recovers most of the gain the latent term does not produce, and that a dense causal next-latent term stacked on next-code AR scores above either component alone at all three budgets.
5. Three diagnostics measured before the grids that were designed against them, each identifying a route by which the masked-span objective can be satisfied without the encoder carrying information a probe would want (Section 6.2).

What this paper is not. It is not a benchmark result. DE-SynPUF is a synthetic public-use file constructed by sampling and swapping fields across real Medicare beneficiaries specifically in order to break re-identifiable associations, so the predictive structure between a patient's history and their future is weakened by construction. The absolute AUROCs below are a property of that source and are not comparable to published numbers on MIMIC-IV or EHRSHOT. What the design supports

are *relative* comparisons at matched compute, on identical rows, with an untrained control and a count-feature ceiling computed on the same rows.

2 Related work

Foundation models for structured EHR. CLMBR-style autoregressive pretraining over serialized clinical codes, together with the EHRSHOT few-shot benchmark, established the evaluation shape this work follows [17]. MOTOR [16] pretrains on time-to-event targets rather than next-code identity. ETHOS [14] and CEHR-GPT [13] pursue generative patient-timeline modelling, the latter with explicit temporal tokens; EHRMamba [8] substitutes a state-space backbone for attention. Delphi-2M [15] models disease progression generatively at population scale. Wornow et al. [18] study what long-context architectures over EHR event streams actually use their context for, which is the question our diagnostics in Section 6.2 ask of a latent objective. Zhang et al. [21] report scaling behaviour for EHR foundation models, and Guo et al. [9] study how tokenization choices trade off against downstream quality — both directly relevant to the vocabulary construction in Section 3 and the token-budget grids in Section 6.5.

Data and task standards. MEDS [2] is the interchange format every source in this project is lowered into; ACES [19] evaluates the cohort and label windows from declarative YAML, so the seven tasks below are specified rather than implemented. MEDS-Tab [12] is the tabularization baseline our count-feature construction deliberately mirrors.

Joint-embedding prediction. I-JEPA [3] introduced masked latent prediction with a stop-gradient target encoder; V-JEPA 2 [4] extends it to video at scale. LeJEPA [5] replaces heuristic anti-collapse machinery with SIGReg, an explicit isotropic-Gaussian penalty estimated through random one-dimensional projections, and is the formulation this implementation follows; Li et al. [11] report that the teacher branch can be frozen without loss, which is the same question our `shared`-versus-`ema` ablation asks in a different form. LeNEPA [6] pursues next-latent prediction for time series without augmentation, which is structurally the objective family (b) below arrives at from a different direction.

JEPA and world models for clinical data. Ennadir et al. [7] apply joint-embedding prediction to general time series. Clin-JEPA [20] pretrains a joint-embedding predictive objective on EHR patient trajectories directly, and Adam et al. [1] argue for a world-model rather than document-modelling framing of the longitudinal record; FoMoH [10] proposes a clinically-grounded evaluation for structured-EHR foundation models. The masked-span variant in Section 4.3 is the closest analogue to that line, and is the variant that does not clear its untrained control at the budgets reported here.

3 Data and representation

3.1 Canonical MEDS layout

Three sources are supported and each is lowered once into one identical on-disk shape, so that no component downstream of the ETL learns that more than one source exists: MIMIC-IV (credentialed, via PhysioNet), CMS DE-SynPUF (public use), and Synthea (generated). Shards carry the MEDS 0.4 core columns — `subject_id`, `time`, `code`, `numeric_value`, `text_value` — sorted by `(subject_id, time, code)` with null times first, so a subject's static facts precede their timeline. Shards are subject-disjoint. Source-specific extension columns are dropped rather than carried; `meds_etl`'s MIMIC output has roughly twenty of them, and keeping them would mean the model sees a different column set per source. Every shard is validated against the MEDS schema before it is written.

Splits are a pure function of the subject identifier: `blake2b("split:<seed>:<subject_id>")` truncated to 64 bits, taken modulo 10,000 and cut at 8000/9000 for an 80/10/10 train/tuning/held-out draw, with the seed pinned at 20240917 by a test. Codes are *not* harmonised across sources — there is no shared ontology mapping step, and inventing one would be guesswork — but the shape `NAMESPACE//value` is.

The source used for every experiment below. All results in this paper are on `desynpuf-s1`: 116,352 subjects and 14.2M events in MEDS form. DE-SynPUF is a claims-derived file. It contains diagnoses (ICD-9-CM), procedures (ICD-9 procedure, HCPCS), drug events (NDC), admissions, discharges with length of stay, DRGs, and beneficiary demographics. **It contains no laboratory results, no vital signs, and no clinical notes.** The set of event types available to the model is therefore narrower than in a typical clinical extract, and the value channel is sparsely populated (length of stay in days, days supplied).

3.2 Tokenization

One MEDS event becomes one token. The vocabulary and the value quantizer are fit on the `train` split only and then reused for `tuning` and `held_out`. Three steps produce the vocabulary:

1. **Normalization.** `NDC//<digits>` is truncated to its first nine characters. An 11-digit NDC is labeler(5)+product(4)+package(2); the package segment records whether the bottle held 30 tablets or 90. Numerically this dominates everything else: 266,653 of DE-SynPUF's 288,274 distinct training codes are NDCs, and folding to nine digits leaves 138,443 distinct codes.
2. **Hierarchical rollup before `[UNK]`.** Ids 0–3 are `[PAD]`, `[UNK]`, `[CLS]`, `[MASK]`. A code with at least 5 training occurrences gets its own id. A rarer code is truncated towards an ancestor that is frequent enough: ICD codes strip one character at a time (`ICD9CM/250.01` → `250.0` → `250` → `25` → `2`), NDC falls back to the 5-digit labeler, HCPCS to its first three characters, anything else to the bare prefix, and only then to `[UNK]`. Admission is one bottom-up sweep in which a node holds its own count plus the mass of its *rejected* children, so mass is never counted at two levels at once.
3. **The cap.** `-max-vocab 30000` bounds the table; entries are demoted smallest-mass-first, each demotion handing its whole mass to the nearest still-admitted ancestor.

On DE-SynPUF at 30,000 rows (from 252,935 uncapped), the `[UNK]` rate is 0.0 on every split — every DE-SynPUF code has a prefix in the table to fall back to — and the fallback-to-ancestor rate is 16.2% train, 16.3% tuning, 16.5% held-out.

Value quantizer. A raw float is meaningless without its code, so quantization is per code. Every vocabulary id with at least 20 training observations gets nine interior decile edges plus a mean and standard deviation; ids below that share their prefix's fit, and failing that a global one. Non-negative codes with $|\text{skew}| > 2$ are $\log(1 + x)$-transformed before the moments are taken. A value sitting exactly on an edge falls in the *lower* bin, which keeps a code whose values are mostly one constant from smearing across bins.

Per-event features. Table 1 lists what the cache stores per event. Subjects with no `MEDS_BIRTH` are anchored at their first event and flagged, so a model can learn not to trust their ages. Events with identical timestamps get $\log \Delta = 0$ and keep their source order.

Feature arrays are written as one flat memory-mapped vector per split with a side table giving each subject's offset and length, so a training run pages in only the windows it samples. `EventSequenceDataset` yields one item per subject; `sampling="random_window"` draws a fresh contiguous window on each request. On this cache the `train` split holds 78,997 subjects after dropping those with fewer than 16 events (13,995 dropped) and 11.3M events; a `max_len` 256 window averages 121 real events (47% fill, measured over 2,000 draws).

Table 1: Per-event features in the tensor cache. One MEDS event becomes one token. `time_min` is not a model input; it is what the evaluation harness binary-searches to cut history at an anchor.

feature	dtype	meaning
`code_id`	int32	vocabulary id, after hierarchical fallback
`value_bin`	int8	0 = no numeric value; 1–10 = decile of the code's training values
`value_z`	float32	z-score of the (optionally $\log 1p$-ed) value, clipped to ± 5; 0 when absent
`age`	float32	years since `MEDS_BIRTH`, clipped to $[0, 120]$
`log_delta`	float32	$\log 1p$ of hours since the subject's previous event; 0 for the first event and for ties
`time_min`	int64	raw event time, minutes since epoch, for label alignment only

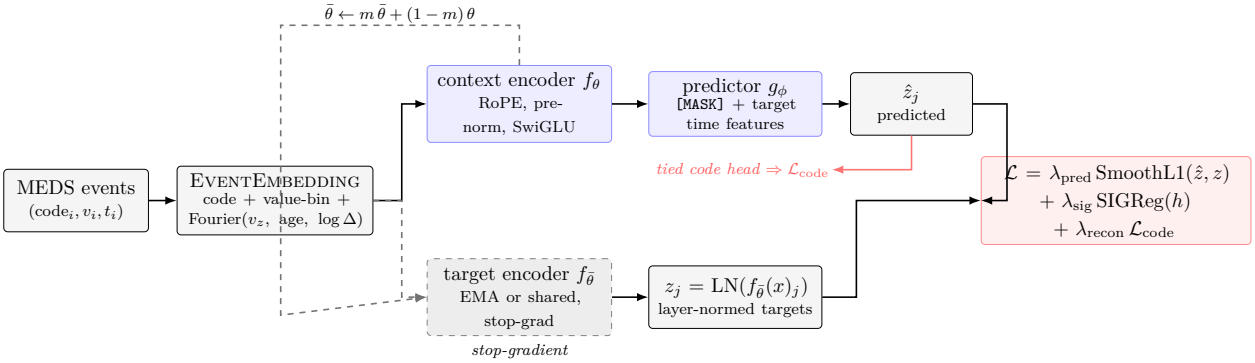

Figure 1: The shared architecture. The context encoder f_θ and the predictor g_ϕ carry gradients; the target encoder $f_{\bar{\theta}}$ runs under stop-gradient, either as a weight-shared copy or as an EMA copy, and is never told what code or value the masked span holds. Nothing is reconstructed in input space by the latent term; the optional code-reconstruction term $\mathcal{L}_{\text{code}}$ passes the *predicted* latent through the tied code head.

4 Model and objectives

4.1 Event embedding

For event i with code id c_i, value bin b_i, standardized value z_i, age a_i and log inter-event gap d_i, the input token is a *sum* — nothing is concatenated:

$$x_i \;=\; E_{\text{code}}[c_i] \;+\; E_{\text{bin}}[b_i] \;+\; \mathbb{1}[b_i \neq 0] \cdot g_v(z_i) \;+\; m_i\big(g_a(a_i) + g_\delta(d_i)\big), \tag{1}$$

followed by dropout. The gate $\mathbb{1}[b_i \neq 0]$ exists because `value_z` is stored as 0.0 for value-less events and 0.0 is also an ordinary z-score; without it, "no value" and "exactly average" would collide. The factor $m_i \sim \text{Bernoulli}(1 - p_{\text{tdrop}})$ is per-token time-feature dropout, applied during training only and only in the cells that use it.

Each $g_\bullet$ is an independent two-layer network over a fixed Fourier featurization of its scalar,

$$\begin{aligned} g(u) \;&=\; W_2\,\text{GELU}(W_1\,\phi(u) + b_1) + b_2, \\ \phi(u) \;&=\; \big[\sin(2\pi f_1 u), \ldots, \sin(2\pi f_K u),\; \cos(2\pi f_1 u), \ldots, \cos(2\pi f_K u)\big], \end{aligned} \tag{2}$$

with $K = 16$ frequencies log-spaced from 10^{-2} to 10^{1} cycles per input unit, so periods span 100 down to 0.1 units, and a hidden width equal to the model width. Note the layout: all sines then all cosines, not interleaved. Time therefore enters the model *twice and differently* — as a *content* feature through g_a and g_δ, and as *position* through rotary embeddings inside attention. That separation is what makes the `notime` ablations of Section 6 possible: the content path can be switched off while the positional path remains.

4.2 Encoder, predictor, target encoder

Encoder. A pre-norm transformer: $x \leftarrow x + \mathrm{Attn}(\mathrm{LN}(x))$ then $x \leftarrow x + \mathrm{MLP}(\mathrm{LN}(x))$, with a final LayerNorm after the last block. Attention uses one fused QKV projection and rotary position embeddings (base 10^4) applied to queries and keys only; positions are *sequence indices*, not wall-clock, since irregular spacing is already carried by g_δ. The feed-forward is SwiGLU, $\mathrm{MLP}(x) = W_3\big((W_1 x) \odot \mathrm{SiLU}(W_2 x)\big)$, at hidden width $8 \cdot \mathrm{round}(\frac{2}{3} \cdot \frac{d r}{8})$ for ratio $r = 4$. Dropout is 0.1; attention dropout is 0. A learned `[CLS]` token is prepended at index 0. Padding is masked key-side only. The encoder runs bidirectionally for the masked-span objectives and causally (a lower-triangular mask intersected with the padding mask) for AR, next-latent and window. One consequence matters downstream: under a causal mask the `[CLS]` output row is a function of the `[CLS]` parameter alone — a prefix token cannot read the sequence without leaking the future back at the next layer — which is why the evaluation probe uses the last valid token rather than `[CLS]` for causal checkpoints.

Predictor (masked-span only). A narrower transformer over one sequence of the encoder's original length, with no `[CLS]`. Each position is one of three things: a *context* position, carrying the encoder output projected to the predictor width; a *target* position, carrying a mask token; or *dropped*, excluded from attention and zeroed. A mask token is

$$t_i \;=\; \texttt{[MASK]} \;+\; P\big(g_a^{\mathrm{pred}}(a_i) + g_\delta^{\mathrm{pred}}(d_i)\big), \tag{3}$$

carrying the *target's* time and nothing else about it; with `mask_token_time: false` the second term is dropped and every target's token is the same vector, distinguished only by rotary position at its original index. Attention inside the predictor is bidirectional over context $\cup$ target, and the output is projected back to encoder width. No target content reaches the predictor by any route: not through the mask tokens, and not through the context representations, because the context encoder pass never attended to target positions. A unit test asserts this numerically by perturbing target codes and values and checking the predictions are bit-identical.

Target encoder. A stop-gradient copy of the embedding and encoder in one of two modes. `shared` evaluates the online weights under `no_grad`: there is no second copy and nothing to schedule, and collapse is prevented by SIGReg alone. `ema` — used by every configuration reported here — keeps a frozen deep copy whose parameters track the online ones,

$$\bar{\theta} \;\leftarrow\; m(t)\,\bar{\theta} \;+\; \big(1 - m(t)\big)\,\theta, \qquad m(t) \;=\; m_0 + (m_1 - m_0)\,\mathrm{clip}\big(\tfrac{t}{T}, 0, 1\big)\,, \tag{4}$$

with buffers copied rather than averaged. The momentum schedule is *linear*, not cosine, from $m_0 = 0.996$ to $m_1 = 1.0$ over the run's step count, so the target network freezes exactly at the end of the run. Stop-gradient is enforced three ways: `requires_grad_(False)` on the copy, `no_grad` on the target pass, and `.detach()` on the returned tensor. Time-feature dropout is set to zero on the target side — it is an augmentation of the online input only.

4.3 The objectives

The total objective is

$$\mathcal{L} \;=\; \lambda_{\mathrm{pred}}\,\mathcal{L}_{\mathrm{lat}} \;+\; \lambda_{\mathrm{sig}}\big(\mathrm{SIGReg}(H_{\mathrm{tok}}) + \mathrm{SIGReg}(H_{\mathrm{cls}})\big) \;+\; \lambda_{\mathrm{recon}}\big(\mathcal{L}_{\mathrm{code}} + \mathcal{L}_{\mathrm{bin}}\big). \tag{5}$$

Note that λ_{sig} multiplies the *sum* of the two SIGReg terms, not their mean. Setting $\lambda_{\mathrm{pred}} = 0$ does not multiply the latent term by zero — it *drops* it, so the target encoder never runs and the EMA update is skipped, which is what makes the `recon_only` cells a genuine removal rather than a reweighting. Setting $\lambda_{\mathrm{sig}} = 0$ yields the `nosig` cells.

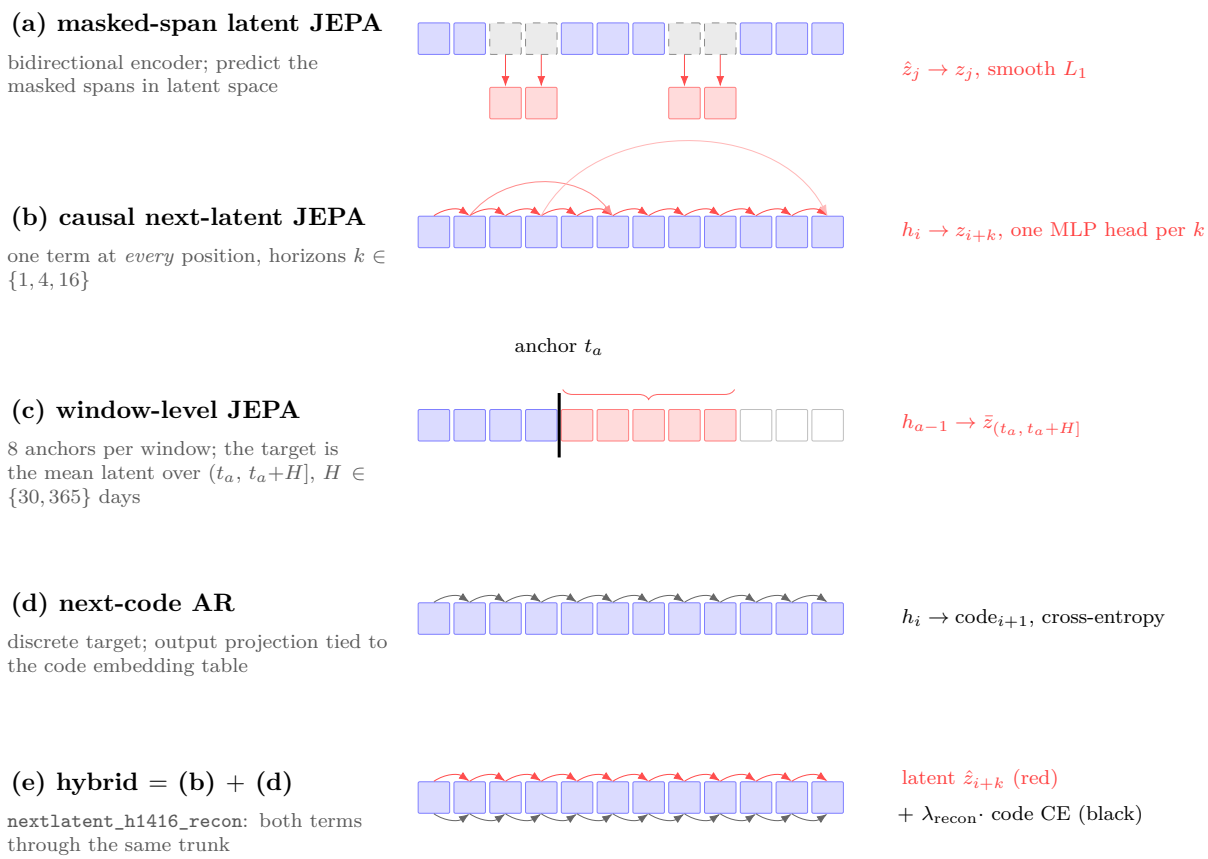

Figure 2: The five objective families, drawn on a common 12-token strip. Blue is observed context, red is a predicted or supervised position, grey dashed is masked. The difference between (a) and (b)–(e) that the experiments turn on is *density*: (a) places a loss term only at the 10–30% of positions a mask covers, while (b), (d) and (e) place one at every position.

(a) Masked-span latent JEPA. Each row of a batch independently draws either a *future-span* mask (with probability p_{future}, default 0.6) or a *multi-block* mask. The future-span mask picks a cut c uniformly on $\{\lceil 0.3L \rceil, \ldots, \lfloor 0.9L \rfloor\}$ and a span length uniformly on $\{8, \ldots, 64\}$, takes $[0, c)$ as context and $[c, c+s)$ as targets, and *drops everything after the span from both*, so the task is "given this history, what does the next stretch of this record look like" rather than an interpolation with both endpoints pinned. The multi-block mask places 2–4 possibly-overlapping blocks of size $\text{round}(L(0.05 + 0.10u))$, $u \sim \mathcal{U}(0, 1)$ per block, takes the complement as context and thins it at a rate drawn uniformly on $[0, 0.3]$. Realized target coverage is typically 10–30% of the window. Given the target set $\mathcal{T}$, the predictor emits $\hat{z}_j$ and the loss is a smooth L_1 against the layer-normed target:

$$\mathcal{L}_{\text{lat}} = \frac{1}{|\mathcal{T}|\, D} \sum_{j \in \mathcal{T}} \sum_{k=1}^{D} \ell_\beta(\hat{z}_{jk} - \tilde{z}_{jk}), \qquad \tilde{z}_j = \text{LayerNorm}\big(\text{sg}[f_{\bar{\theta}}(x)_j]\big), \tag{6}$$

where ℓ_β is the Huber/smooth-L_1 function with $\beta = 1.0$, the reduction is a mean over rows *and* feature dimensions, and both tensors are cast to fp32 regardless of autocast. The target LayerNorm carries *no learned affine* and lives in the loss rather than in the model: it is what forbids the trivial solution of shrinking targets toward zero, so the scale of the target representation cannot be gamed.

Two flags vary what the target encoder is shown. `target.span_only` runs it on the target positions compacted into their own sequence with their own attention mask and their own `[CLS]`, so that a target latent cannot contain the context the predictor was already handed; the honest cost is that rotary position then sees within-span offsets, so the gaps *between* multi-block targets are lost. `target.time_features` removes the age and gap terms from the target encoder's embedding, making a target latent a function of content alone; the cells that use it also apply `time_feature_dropout` 0.3 to the online encoder, so under EMA the online weights are not handed an input distribution they

have never seen.

(b) Causal next-latent JEPA. One head per horizon $k \in \mathcal{H}$, each a two-layer GELU MLP at the *encoder's own* width — deliberately not the transformer predictor (there is no set of masked positions to attend over, only one context vector per prediction) and deliberately not wider, so that representation quality cannot be offloaded into the head. At every position i where both i and $i+k$ are real events, the head input is $x_i^{(k)} = h_i + g_a^{\text{pred}}(a_{i+k}) + g_\delta^{\text{pred}}(d_{i+k})$ — the *time* of the event being predicted and nothing else about it — and

$$\mathcal{L}_{\text{lat}} \;=\; \frac{1}{|\mathcal{H}'|} \sum_{k \in \mathcal{H}'} \underbrace{\frac{1}{|\mathcal{V}_k|\, D} \sum_{i \in \mathcal{V}_k} \sum_{j=1}^{D} \ell_\beta\Big(\big[g^{(k)}(x_i^{(k)})\big]_j - \tilde{z}_{(i+k)j} \Big)}_{\mathcal{L}^{(k)}}, \qquad \mathcal{H}' = \{k \in \mathcal{H} : |\mathcal{V}_k| > 0\}. \tag{7}$$

The outer average is over *horizons*, not over the pooled rows: a single smooth-L_1 over the concatenation would weight k=1 most and k=16 least, since $|\mathcal{V}_k| = L - k$. Separate heads rather than one head with a horizon embedding make "horizon 4 learned nothing" a readable statement about one module. This objective has as many terms per window as the AR loss. The risk, stated in the protocol before the grid ran, is that the target encoder is causal too, so z_{i+1} summarises a prefix h_i already knows and a predictor can score well by copying; the per-step diagnostic is cos_gap, the mean cosine of predictions with their own targets minus with shuffled ones.

(c) Window-pooled latent JEPA. Eight anchors per window are drawn without replacement from $[\lceil 0.3L \rceil, \lfloor 0.9L \rfloor)$ — the same band the future-span mask draws its cut from, for the same reason: too early and there is no history to summarise, too late and there is no future left inside the window. The context summary for anchor a is the causal encoder's output at $a-1$, so "strictly before the anchor" is enforced by the attention mask rather than by a second forward pass over a truncated window, which would cost K times the compute for a context differing only in direction. The prediction is $\hat{z}_{a,H} = \text{MLP}(h_{a-1} + u_H)$ with u_H a learned horizon embedding — here *one* shared head plus a horizon embedding, the opposite choice from (b), because these horizons are a scale rather than separate tasks. The target is the mean target latent over the events inside the horizon:

$$\bar{z}_{a,H} \;=\; \frac{1}{|\mathcal{W}_{a,H}|} \sum_{j \in \mathcal{W}_{a,H}} \tilde{z}_j, \qquad \mathcal{W}_{a,H} = \{\, j : \text{valid}_j \wedge\; t_a < t_j \le t_a + H \,\}, \tag{8}$$

with times compared in int64 minutes since epoch throughout, because minutes-since-epoch passes 2^{24} in 2001 and float32 could not separate events a minute apart. An anchor/horizon pair is skipped when the horizon end runs past the last event in the window — the future is not observed there, and a truncated pool would be indistinguishable from a genuinely quiet one — or when no event falls inside it; the realized skip fraction is logged every step. Measured on this cache at `max_len` 256 with 8 anchors before the grid was queued: at $H = 30$ days, 0.71 of anchors are kept with 6.5 events per kept pool; at $H = 365$ days, 0.47 are kept with 44 events per pool.

(d) Next-code AR. The causal encoder's output at position i is projected through a head whose output matrix *is* the code embedding table (weight tying) to a softmax over the 30,000-code vocabulary:

$$\mathcal{L}_{\text{code}} \;=\; -\frac{1}{N} \sum_i \log \frac{\exp\Big(E_{\text{code}}[c_{i+1}]^\top h_i\Big)}{\sum_{c'} \exp(E_{\text{code}}[c']^\top h_i)}. \tag{9}$$

Position i scores only when both it and its successor are real events, so the last valid position of every window is dropped; `[PAD]` doubles as the ignore index. Only `code_id` is predicted — value and time remain inputs, never targets. The softmax is chunked at 2,048 rows under gradient checkpointing, which leaves the arithmetic unchanged (pinned by a test) while cutting peak memory for a 30,000-way softmax over ∼8,000 positions from 13.3 GB to well under 1 GB.

The same tied head serves as the auxiliary code-reconstruction term $\mathcal{L}_{\text{code}}$ for the other objectives, but *what it reads differs by family*, and this matters for interpreting the results. For masked-span JEPA and for the window objective it is applied to the *predicted latent*: a predicted latent that must name its event's code cannot be a code-free summary, and the gradient reaches the encoder through the context. For the next-latent objective it is applied to the *encoder's own* hidden states, making it the AR loss verbatim. The masked-span variant carries a second term $\mathcal{L}_{\text{bin}}$, a cross-entropy over the 11 value-bin classes at the same weight. For the window objective the term is instead a multi-label binary cross-entropy from the predicted pooled latent to the multi-hot set of distinct codes inside that anchor's horizon,

$$\mathcal{L}_{\text{BCE}} = \frac{1}{NV} \sum_{n=1}^{N} \sum_{v=1}^{V} \text{BCE}(\text{logit}_{nv},\, y_{nv})\,, \tag{10}$$

reduced as a mean over rows *and* classes, so the term starts at $\log 2$ and is dominated by the roughly 30,000 absent codes rather than by the few dozen present ones. That is the standard formulation and the one $\lambda_{\text{recon}} = 0.1$ was chosen against; a positive-class reweighting would be a different experiment.

(e) The hybrid. `nextlatent_h1416_recon` is (b) at horizons $\mathcal{H} = \{1, 4, 16\}$ plus (d) through the same tied head and the same trunk, at $\lambda_{\text{pred}} = 1.0$, $\lambda_{\text{sig}} = 0.05$, $\lambda_{\text{recon}} = 0.1$, causal, EMA target. Because λ_{recon} here *is* the AR loss read off the encoder, setting $\lambda_{\text{pred}} = 0$ reduces the hybrid to next-code AR exactly, which is pinned by a unit test. This is what makes the hybrid readable as a controlled addition to AR rather than as a different model. All hybrid cells reported through Section 6.6 use $\lambda_{\text{sig}} = 0.05$; Section 6.7's knob ablation finds $\lambda_{\text{sig}} = 0$ (no SIGReg) the best-scoring variant at both seeds, and the repository's default configuration (`configs/pretrain_default.yaml`) drops the term accordingly.

4.4 Anti-collapse: SIGReg

Following LeJEPA [5], collapse is prevented by an explicit distributional penalty rather than by architectural asymmetry, predictor stop-gradients or negative pairs. A batch of embeddings $Z \in \mathbb{R}^{N \times D}$ is projected onto K directions drawn uniformly on the unit sphere *afresh at every call* — every step, and independently for the token and `[CLS]` terms — giving $P = ZA$, and each resulting one-dimensional empirical distribution is penalized by an Epps–Pulley statistic, the integrated squared difference between its empirical characteristic function and that of a standard normal:

$$\begin{aligned} S_k(t) &= \left(\tfrac{1}{N}\textstyle\sum_n \cos(tP_{nk}) - e^{-t^2/2}\right)^2 + \left(\tfrac{1}{N}\textstyle\sum_n \sin(tP_{nk})\right)^2, \\ \text{SIGReg}(Z) &= \frac{1}{K}\sum_{k=1}^{K} \int_{-t_{\max}}^{t_{\max}} S_k(t)\, e^{-t^2/\sigma^2}\, \mathrm{d}t, \end{aligned} \tag{11}$$

with the integral evaluated by the trapezoid rule on a 17-point uniform grid over $[-5, 5]$ and $\sigma = 1$. The configurations reported here use $K = 256$ directions. Cost is $O(ND)$ — nothing pairwise.

Two deviations from the textbook Epps–Pulley test are deliberate and are load-bearing. First, the N prefactor is *omitted*: it is what makes the statistic converge to a fixed null distribution, but as a loss it would multiply the regularizer by the row count, and at the 8,192 token rows this implementation subsamples to, that would swamp the prediction term at $\lambda_{\text{sig}} = 0.05$. Second, *nothing is standardized inside SIGReg* — no centering, no whitening, no per-dimension normalization. The embeddings must become isotropic Gaussian on their own; subtracting the mean would hand the model the part of the objective it is supposed to learn.

The term is applied at two granularities, and both are needed: an encoder can produce perfectly isotropic token outputs while every subject embedding lands in the same place. The token term reads the context positions the gradient-carrying encoder actually attended to, subsampled without replacement to at most 8,192 rows; the `[CLS]` term reads exactly `batch_size` rows. Its purpose here

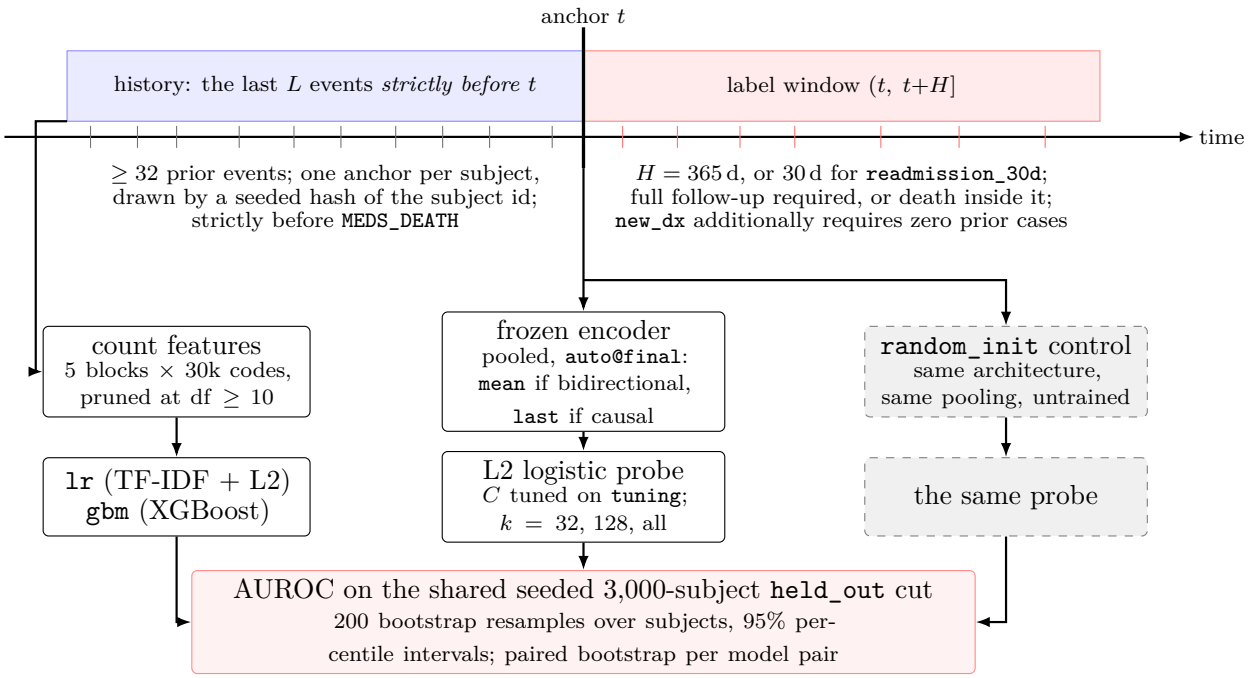

Figure 3: The evaluation protocol. Every model in a run — count baselines, frozen-encoder probes, and the untrained control — is fit and scored on identical anchors and identical rows, which is what makes the paired bootstrap legitimate and stops a checkpoint from looking better on an easier subset.

is specific: without it, the masked-span target embeddings are 45% between-window variance and a plain context-mean baseline reaches cosine 0.73 to the targets (Section 6.2).

Four collapse diagnostics are logged every step but never optimized: the mean per-dimension standard deviation of valid token rows, the effective rank $\exp(-\sum_i p_i \log p_i)$ over normalized singular values of a mean-centred subsample, and the cosine of predictions with their own versus mismatched targets, whose difference is cos_gap.

5 Evaluation protocol

5.1 Tasks

Seven binary tasks, defined in ACES YAML from a shared predicate file so that cohorts and label windows are declarative rather than implemented. Labels come from the MEDS extract, never from the tensor cache — the cache rolls rare codes up to an ancestor, so `ICD9CM//25001` and `ICD9CM//25002` can share an id there, and a label built on cache ids would not be the label its name claims.

- `mortality_365d` — a `MEDS_DEATH` event in $(t,\ t + 365\text{d}]$.
- `inpatient_365d` — an `ADMISSION//INPATIENT` in $(t,\ t + 365\text{d}]$.
- `readmission_30d` — anchored on an inpatient *discharge*; another inpatient admission in $(t,\ t + 30\text{d}]$.
- `new_dx_365d/{ckd, copd, diabetes, heart_failure}` — a first-ever occurrence of the condition in $(t,\ t + 365\text{d}]$, restricted to subjects with *zero* occurrences at or before the anchor. On DE-SynPUF the matchers are the ICD-9-CM prefixes 585, {490, 491, 492, 494, 496} (asthma, 493, is deliberately excluded), 250 and 428 respectively.

Every target window is `start_inclusive: false, end_inclusive: true`, i.e. the half-open interval $(t,\ t + H]$: the anchor event itself lies in neither the history nor the label window.

5.2 Anchors and leakage rules

One anchor per subject. Candidate times are the subject's distinct event times, filtered in order by: (i) bearing the task's anchor concept, where it has one (only `readmission_30d` does); (ii) at least 32 events strictly before; (iii) strictly before `MEDS_DEATH`, if the subject has one; and (iv) either a full horizon of record afterwards or death inside the horizon — otherwise a negative label would be censoring, not a negative. Among survivors the kept time is chosen by `blake2b("anchor:<seed>:<subject_id>")` modulo the candidate count, so the draw is a pure function of (seed, subject_id) and is independent of row order, shard count and library version. The anchor seed is 20260903.

Everything a model sees passes through one choke point, `EventSequenceDataset.windows_at`, which binary-searches the cache's `time_min` with `side="left"`: an event whose timestamp equals the anchor is excluded along with everything after it. Because anchors are strictly before `MEDS_DEATH` and history is strictly before the anchor, no event at or after death can enter a history window. The test suite builds the cohort twice, the second time with a `MEDS_DEATH` inserted one day after each held-out subject's anchor — an event that flips `mortality_365d` from 0 to 1 — and asserts the count features and the encoder embeddings come out bit-identical.

5.3 Baselines, probes and controls

Count-feature baselines. Five blocks of `vocab_size` columns each — $\log 1p$ counts over the last 30 days, 365 days and all history, the most recent `value_z` per code, and a $\log 1p$ numeric-observation count — giving roughly 150,000 columns, pruned to those nonzero in at least 10 *training* rows (103k–110k kept per task). `lr` is L2 logistic regression with C tuned on the `tuning` split over $(0.003, 0.01, 0.03, 0.1, 0.3, 1.0)$; sparse inputs are scaled with a TF-IDF transform fit on `train` only.[1] `gbm` is XGBoost with `tree_method="hist"`, selected on `tuning` AUROC from three settings, fit in a subprocess that never imports torch (both libraries ship their own OpenMP runtime, and on macOS arm64 whichever loads second corrupts the first).

Frozen-encoder probes. The encoder runs in `eval()` mode, fp32, under `no_grad`, with no masking, over the last `max_len` events strictly before the anchor. The pooled representation is fed to the same L2 logistic regression, C tuned on `tuning`. Nothing is fine-tuned, so a difference between two checkpoints is a difference in the representation. Pooling is resolved per architecture: `mean` over valid tokens for a bidirectional encoder, the final *valid* token for a causal one. This matters for cross-grid comparison: grid 1 probed every cell at `cls_mean@final`, while grids 2–5 and both scale grids use the architecture-resolved default. A causal encoder's `[CLS]` row is a function of the `[CLS]` parameter alone — it sits at index 0 and attends only to itself — so half of its `cls_mean` vector carries nothing.

Untrained controls. `random_init` rebuilds the checkpoint's own architecture from its own stored `model_config` and skips `load_state_dict`, then runs the identical probe at the identical pooling. A pretrained checkpoint that does not beat its own untrained twin has not learned anything this probe can use. One control is computed per (architecture, pooling) pair, not per cell.

Held-out cut and intervals. The evaluation split is capped at a seeded 3,000 subjects by the same `blake2b` construction as the anchor draw, so a subject kept for one task is kept for every task in which it appears; `train` and `tuning` are untouched, so every model is still fit on the full training data. Metrics are AUROC, AUPRC, Brier and calibration slope, each with a 95% percentile bootstrap over subjects at 200 resamples; resampled draws that are degenerate for a ranking metric are dropped

[1] An earlier version scaled with `StandardScaler(with_mean=False)`, which divides each column by its own standard deviation; because most kept columns are rare, this inflated their scale and forced the tuned C to the low edge of the grid on every task, capping `lr` at 0.55–0.59 AUROC — below the untrained control. The scaling was replaced and only `lr` refit; a regression test reproduces the failure shape. This is recorded because the uncorrected numbers appear in an earlier committed run.

Table 2: Cohort sizes and held-out prevalence for `desynpuf-s1`, before the seeded 3,000-subject evaluation cut. *Anchored* is subjects surviving the anchor rule; *labelled* is those remaining after the task's own inclusion criterion (for `new_dx`, having no prevalent case at or before the anchor). After the cut every task's held-out set is exactly 3,000 subjects.

task	anchored	labelled	held-out n	held-out rate
`mortality_365d`	72,835	72,835	7,358	0.0190
`inpatient_365d`	72,835	72,835	7,358	0.2098
`readmission_30d`	30,592	30,592	3,116	0.0501
`new_dx_365d/diabetes`	72,835	49,293	4,939	0.2264
`new_dx_365d/heart_failure`	72,835	61,024	6,164	0.1308
`new_dx_365d/ckd`	72,835	63,999	6,487	0.0999
`new_dx_365d/copd`	72,835	59,999	6,009	0.1340

rather than imputed, and the surviving count is reported. Every unordered model pair additionally receives a *paired* bootstrap on the same resampled index, so the interval on a difference is not inflated by shared sampling variance.

Few-shot. For $k \in \{32, 128\}$, k positives and k negatives are drawn without replacement from `train`, a probe is fit, and it is scored on the same held-out split; five draws per (task, k, checkpoint). The tuning split is not subsampled. `gbm` has no few-shot fits: it is fit out of process and its returned object holds only the predictions for the matrix given at fit time, so a few-shot refit would score against the wrong matrix.

5.4 Cohort sizes

Table 2 gives the funnel and prevalences before the 3,000-subject cut. Anchoring keeps 72,835 of 116,352 subjects for the 365-day tasks; the follow-up requirement removes more subjects (15,289) than the history requirement (28,156) plus the death rule (72).

6 Experiments

6.1 Pilot grids 1–4: 20 cells at a matched 48M-token budget

Every cell in Table 3 shares `configs/pretrain_pilot.yaml` (4-layer, 192-wide encoder; 2-layer, 96-wide predictor; SwiGLU; dropout 0.1), the `desynpuf-s1` `train` split, 48,005,120 nominal token slots (2,930 steps at batch 64 × `max_len` 256), seed 0, and the same seeded 3,000-subject held-out cut with 200 bootstrap resamples across all seven tasks. Training was on an Apple M4 (MPS). At this budget a cell sees ≈22.7M real events, or about 2.0 epochs of the 11.3M-event train split. *Gain* is a cell's mean AUROC across the seven tasks minus the same mean for its own architecture- and pooling-matched `random_init` control.

Two pairs of control rows land on numerically identical values. `random_init@jepa_notime` and `random_init@jepa_recon_notime` are the same untrained 4 × 192 bidirectional encoder at seed 0 under `mean@final` pooling; `random_init@nextlatent_h1` and grid 2's `random_init@ar_last` are the same untrained causal encoder under `last@final`. A probe reads only the embedding and the encoder, and neither grid's reconstruction head nor its `notime` flag exists at initialisation. Both pairs are computed independently in their own grids rather than reused, so their agreeing is a determinism check.

What each grid asked. Grid 1 asked whether JEPA beats AR at matched compute and whether either beats an untrained control, sweeping target mode, SIGReg weight and masking mix one at a time off `jepa_ema`. Grid 2 asked whether four probe-independent shortcuts explain grid 1's null result, closing a time-conditional prior, a discarded-code-identity route and a context-copy route,

Table 3: Pilot grids 1–4: held-out AUROC by task, all 20 trained cells and their four controls, seed 0, 48M nominal token slots each. Sorted by mean AUROC descending. *Gain* is against the row's own control; it is "—" for the count-feature baselines and for the control rows themselves. Grid-1 rows were probed at `cls_mean@final` and grid 2–4 rows at the architecture-resolved default, so a grid-1 number is not directly comparable to a grid 2–4 number for the same cell — but every gain is against a control probed the same way as its cell.

cell	gr.	family	control	inpatient	mortality	ckd	copd	diabetes	heart fail.	readmiss.	gain	mean
`nextlatent_h1416_recon`	4	nextlatent	`@nextlat_h1`	0.7417	0.6332	0.7531	0.7487	0.7618	0.7642	0.6985	+0.0923	**0.7287**
`gbm`	—	count	—	0.7440	0.5723	0.7654	0.7674	0.7721	0.7893	0.6724	—	0.7261
`ar`	1	ar	`@ar`	0.7265	0.6060	0.7586	0.7552	0.7590	0.7437	0.6946	+0.0445	0.7205
`jepa_recon_notime`	3	jepa+recon	`@jepa_r_nt`	0.7043	0.6492	0.7116	0.7173	0.7538	0.7144	0.6714	+0.0296	0.7031
`jepa_content_recon`	2	jepa+recon	`@jepa_nt`	0.7085	0.6265	0.7167	0.7141	0.7432	0.7154	0.6747	+0.0263	0.6999
`recon_only`	3	recon-only	`@jepa_r_nt`	0.7077	0.6068	0.7193	0.7155	0.7441	0.7140	0.6854	+0.0254	0.6990
`recon_only_notime`	3	recon-only	`@jepa_r_nt`	0.7009	0.6173	0.7216	0.7117	0.7445	0.7069	0.6804	+0.0240	0.6976
`jepa_notime`	2	masked-span	`@jepa_nt`	0.6920	0.6332	0.7121	0.7167	0.7477	0.7089	0.6590	+0.0221	0.6957
`lr`	—	count	—	0.7077	0.5537	0.7361	0.7258	0.7397	0.7438	0.6578	—	0.6949
`jepa_recon`	2	jepa+recon	`@jepa_nt`	0.7078	0.5954	0.7169	0.7112	0.7410	0.7122	0.6611	+0.0187	0.6922
`jepa_recon_nosig`	3	jepa+recon	`@jepa_r_nt`	0.6947	0.5923	0.7110	0.7138	0.7473	0.7092	0.6703	+0.0177	0.6912
`jepa_ema_nosig`	1	masked-span	`@jepa_ema`	0.6927	0.6409	0.6968	0.6997	0.7379	0.7031	0.6597	+0.0080	0.6901
`jepa_ema`	1	masked-span	`@jepa_ema`	0.6798	0.6096	0.6944	0.7059	0.7418	0.7027	0.6460	+0.0008	0.6829
`random_init@jepa_ema`	1	control	—	0.6850	0.6021	0.7005	0.6933	0.7264	0.7001	0.6675	—	0.6821
`jepa_content`	2	masked-span	`@jepa_nt`	0.6914	0.5998	0.6984	0.7010	0.7307	0.7019	0.6488	+0.0081	0.6817
`jepa_shared_sig`	1	masked-span	`@jepa_ema`	0.6784	0.6573	0.6865	0.6948	0.7292	0.6955	0.6293	−0.0006	0.6816
`jepa_ema_future`	1	masked-span	`@jepa_ema`	0.6819	0.5837	0.6943	0.7048	0.7382	0.7000	0.6612	−0.0015	0.6806
`jepa_ema_block`	1	masked-span	`@jepa_ema`	0.6763	0.5859	0.6939	0.7035	0.7295	0.7097	0.6442	−0.0046	0.6776
`random_init@ar`	1	control	—	0.6751	0.6160	0.6940	0.6890	0.7211	0.6904	0.6464	—	0.6760
`jepa_content_future`	2	masked-span	`@jepa_nt`	0.6784	0.5776	0.6947	0.6975	0.7277	0.6960	0.6528	+0.0014	0.6750
`random_init@jepa_notime`	2/3	control	—	0.6775	0.5699	0.6962	0.6961	0.7252	0.6882	0.6619	—	0.6736
`window_30_365_recon`	4	window	`@nextlat_h1`	0.6764	0.5944	0.6805	0.7054	0.7274	0.6951	0.6209	+0.0350	0.6714
`nextlatent_h1416`	4	nextlatent	`@nextlat_h1`	0.6687	0.6063	0.6886	0.6823	0.7198	0.6868	0.6401	+0.0340	0.6704
`window_30_365`	4	window	`@nextlat_h1`	0.6725	0.5860	0.6859	0.7028	0.7279	0.6901	0.6259	+0.0337	0.6702
`nextlatent_h1`	4	nextlatent	`@nextlat_h1`	0.6601	0.6108	0.6829	0.6854	0.7198	0.6867	0.6225	+0.0305	0.6669
`random_init@nextlat_h1`	4	control	—	0.6469	0.5339	0.6580	0.6620	0.7100	0.6660	0.5781	—	0.6364

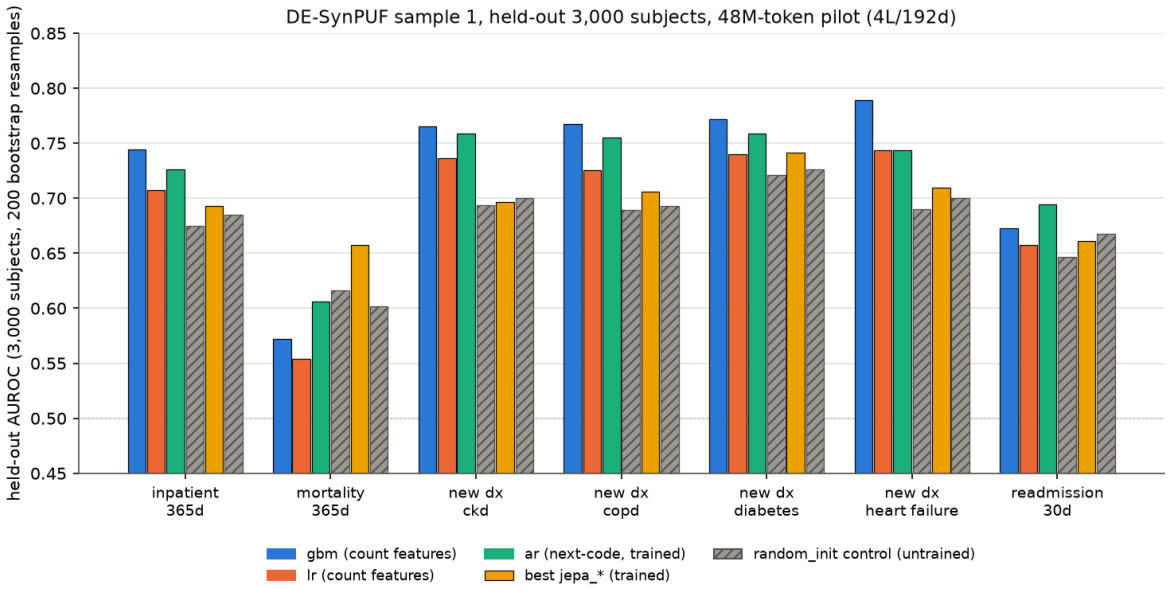

Figure 4: Held-out AUROC by task for `gbm`, `lr`, `ar`, the best `jepa_*` variant and their `random_init` controls, at the 48M-token pilot budget on DE-SynPUF sample 1.

with and without an auxiliary code term. Grid 3 asked whether the latent smooth-L_1 term still adds anything once code reconstruction is present: `recon_only`, with $\lambda_{\text{pred}} = 0$ and no latent term at all, scores 0.6990, which is 0.0041 AUROC below `jepa_recon_notime` at 0.7031. Grid 4 asked whether dense or window-pooled latent prediction transfers, alone and with code supervision.

Two grid-2 rows are excluded from Table 3: `ar_last` and `jepa_ema_mean` train nothing — they re-probe grid 1's checkpoints under a different pooling, and including them would double-count two trained cells as four.

6.2 Diagnostics

Four measurements were taken on grid 1's checkpoints, before grid 2 was designed against them. Each identifies a route by which the masked-span objective can be satisfied without the encoder carrying anything a probe would want. These were one-off measurements; unlike everything else in this paper they do not have committed code in the repository, only their numbers and prose definitions.

1. **The task is mostly a time-conditional prior.** A ridge map from the predictor's mask-token time features *alone* reaches $R^2 = 0.58$ on the layer-normed targets; the trained predictor, which additionally sees the context, reaches $R^2 = 0.79$. Most of what the predictor produces is recoverable from "what time is it" without looking at the patient. The flag that closes this is `predictor.mask_token_time: false`, which reduces a mask token to the bare learned `[MASK]` embedding with rotary position at the target's original index as the only positional information left.

2. **The encoders discard code identity.** A linear probe recovering a token's own `code_id` from the frozen encoder output scores top-1 0.50 on the JEPA checkpoints, against **0.60 for the untrained encoder** and 0.71 for the AR one. Pretraining made this *worse* than initialisation.

3. **Full-sequence targets admit a context-copy shortcut.** Without SIGReg the target embeddings are 45% between-window variance, and a plain context-mean baseline reaches cosine 0.73 to the targets. The target encoder attends over the whole window, including the context the predictor was handed, so a target latent can contain what the predictor already has. The flag that closes this is `target.span_only`.

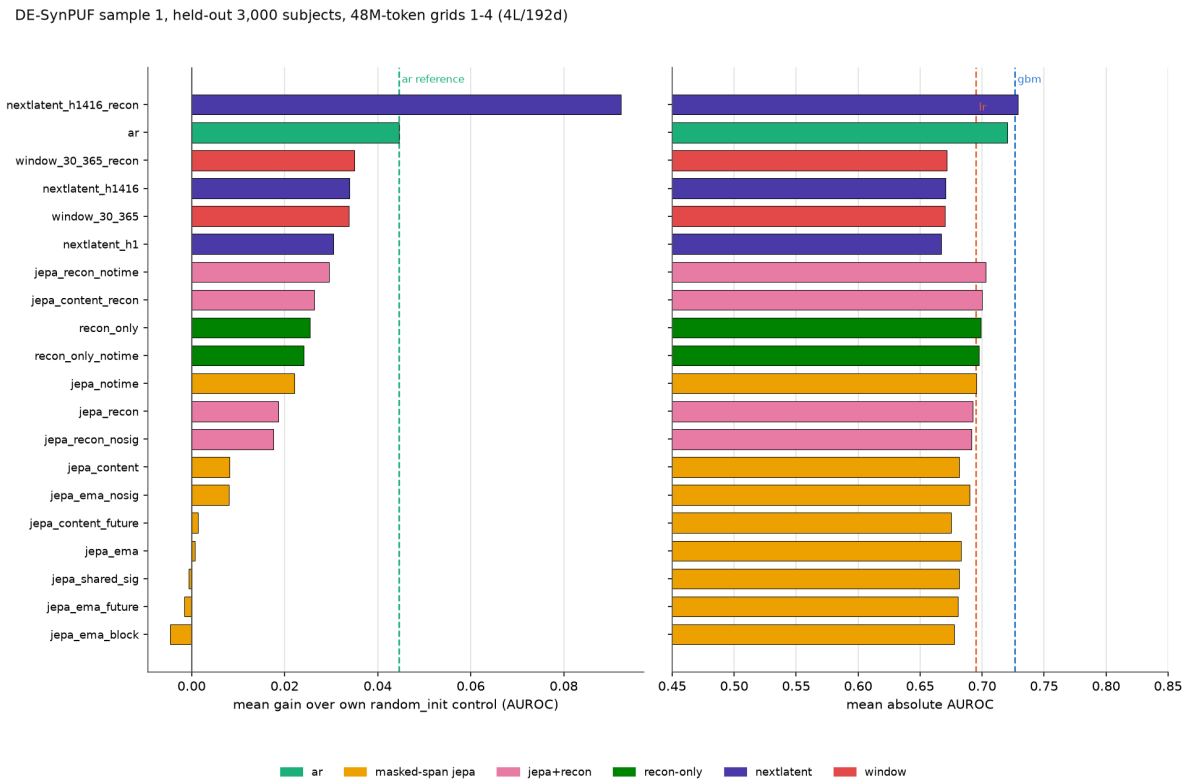

Figure 5: Mean gain over each cell's own `random_init` control (left) and mean absolute AUROC (right), by cell and objective family, across pilot grids 1–4. Regenerated by `scripts/plot_grids.py`, which parses the four committed `summary.md` tables directly.

4. **One diagnostic did not survive re-measurement.** Pooling was initially reported to be costing the causal arm 2.5–3.3 AUROC points (`last@final` against `cls_mean@final` on the AR checkpoint). Re-scored on the full seven-task 3,000-subject held-out cut, `last` wins four tasks of seven, ranges from −2.07 to +2.77 points, and averages +0.17. The pooling default was still changed, on the architectural argument given in Section 5, but the *size* of the effect is not what the diagnostic claimed. It is recorded here because the original figure appears in an earlier committed protocol document.

Read against Table 3: closing the shortcuts moves the probe, but not by much and not to where AR sits. `jepa_notime` gains +0.0221 over its control against `jepa_ema`'s +0.0008; adding the code term takes `jepa_content_recon` to +0.0263; but `recon_only`, which has no latent term at all, retains +0.0254 of that. A cell that closes a shortcut and does not move the probe has said something — that the shortcut was not what was limiting the representation.

6.3 Grid 5: seed replication at 48M tokens

Grid 1's `ar` and grid 4's hybrid were retrained at seeds 1 and 2, same base config, same budget, same cache, same held-out subset and bootstrap protocol. Table 4 gives the three-seed mean and standard deviation per task.

The comparison this table supports is narrow. Per task, the `ar`-versus-hybrid gap is *smaller* than at least one of the two families' own seed ranges on `inpatient_365d`, `new_dx_365d/ckd`, `new_dx_365d/diabetes` and `new_dx_365d/heart_failure`. It exceeds both ranges on only three tasks: `mortality_365d`, `new_dx_365d/copd` and `readmission_30d`. On `copd` the sign is in `ar`'s favour.

Table 4: Grid 5: three-seed mean ± population standard deviation and range of held-out AUROC, at the 48M-token budget. Three-seed mean across the seven tasks: `ar` 0.7234, hybrid 0.7279.

task	`ar` mean ± std	`ar` range	hybrid mean ± std	hybrid range
`inpatient_365d`	0.7417 ± 0.0027	[0.7392, 0.7455]	0.7445 ± 0.0025	[0.7417, 0.7479]
`mortality_365d`	0.6034 ± 0.0061	[0.5951, 0.6094]	0.6219 ± 0.0109	[0.6072, 0.6332]
`new_dx_365d/ckd`	0.7480 ± 0.0044	[0.7419, 0.7514]	0.7525 ± 0.0018	[0.7501, 0.7543]
`new_dx_365d/copd`	0.7604 ± 0.0009	[0.7592, 0.7613]	0.7515 ± 0.0023	[0.7487, 0.7544]
`new_dx_365d/diabetes`	0.7618 ± 0.0032	[0.7574, 0.7652]	0.7643 ± 0.0033	[0.7618, 0.7689]
`new_dx_365d/heart_failure`	0.7660 ± 0.0069	[0.7562, 0.7714]	0.7636 ± 0.0041	[0.7584, 0.7683]
`readmission_30d`	0.6824 ± 0.0064	[0.6738, 0.6890]	0.6969 ± 0.0031	[0.6926, 0.6996]

Table 5: Few-shot held-out AUROC. `gbm` is omitted: it has no few-shot fits in the current evaluation path (Section 5); its full-split numbers are in Table 3.

task	`lr` k=32	`lr` k=128	`ar` k=32	`ar` k=128	hyb. k=32	hyb. k=128
`inpatient_365d`	0.5912	0.6575	0.6707 ± 0.0015	0.7058 ± 0.0045	0.6591 ± 0.0012	0.6976 ± 0.0023
`mortality_365d`	0.5265	0.5332	0.5510 ± 0.0019	0.5668 ± 0.0071	0.5636 ± 0.0028	0.5942 ± 0.0037
`new_dx_365d/ckd`	0.6320	0.6701	0.6739 ± 0.0017	0.7134 ± 0.0048	0.6758 ± 0.0020	0.7035 ± 0.0036
`new_dx_365d/copd`	0.6281	0.6666	0.6840 ± 0.0042	0.7176 ± 0.0011	0.6696 ± 0.0031	0.7036 ± 0.0025
`new_dx_365d/diabetes`	0.6838	0.7031	0.6940 ± 0.0051	0.7356 ± 0.0018	0.6967 ± 0.0021	0.7324 ± 0.0040
`new_dx_365d/heart_failure`	0.6483	0.6663	0.6934 ± 0.0056	0.7227 ± 0.0050	0.6876 ± 0.0017	0.7042 ± 0.0026
`readmission_30d`	0.5680	0.5880	0.5766 ± 0.0078	0.5994 ± 0.0062	0.6100 ± 0.0037	0.6487 ± 0.0009

6.4 Few-shot

Table 5 scores the same six checkpoints from grid 5, plus `lr`, at $k = 32$ and $k = 128$ (k positives and k negatives) as well as the full split. Averaged across the seven tasks: `lr` 0.6111 / 0.6407 / 0.6949; `ar` 0.6491 / 0.6802 / 0.7234; hybrid 0.6518 / 0.6835 / 0.7279. The `ar` and hybrid ± values are the spread over three *training* seeds of each checkpoint's own mean over five few-shot-sample seeds; `lr`'s is the spread over its five few-shot seeds directly, since it has one model and no training-seed axis. `lr`'s ± reads 0.0000 at three decimal places throughout.

Both encoder families are above `lr` at every k on every task. The hybrid is above `ar` at both few-shot sizes on `mortality_365d`, `readmission_30d` and (at k=32) `ckd` and `diabetes`; `ar` is above the hybrid at both sizes on `inpatient_365d`, `copd` and `heart_failure`. On `readmission_30d` at k=128 the hybrid reads 0.6487 against `ar`'s 0.5994.

6.5 Scaling to 200M and 1B tokens

Two grids were run on an RTX 4060 (8 GB) with `configs/pretrain_scale.yaml`: a 6-layer, 256-wide encoder with a 4-layer, 128-wide predictor, `max_len` 512, batch 64, bf16, measured at roughly 33k tokens/s. The held-out subset, anchors, bootstrap settings and seven tasks are identical to the pilot grids. Table 6 gives every trained cell across all three budgets.

`random_init@ar` is 0.6547 mean AUROC at both 200M and 1B (the same untrained 6×256 causal encoder at the same seed); `random_init@hybrid` at 1B is numerically identical to it, for the same reason the pilot control pairs agree. `random_init@jepa_ema` at 200M is 0.6742.

The `ar` row at 1B was re-evaluated after a fix to the evaluation harness. Before commit `8721a62`, a checkpoint model's embedding cache was keyed on its display name alone, so a second grid whose cell was also named `ar` shared one cache file with the 200M grid's `ar` and would have silently reported that grid's numbers. The cache key now includes a content fingerprint of the checkpoint file itself.

Measured behaviour across budgets. `ar` goes 0.7205 → 0.7304 → 0.7251 (1B a 3-seed mean). It does not improve from 200M to 1B (−0.0053) and drops on six of seven tasks over that step — every task but `mortality_365d` (0.5946 → 0.5997, +0.0051): `inpatient` 0.7515 → 0.7452, `ckd`

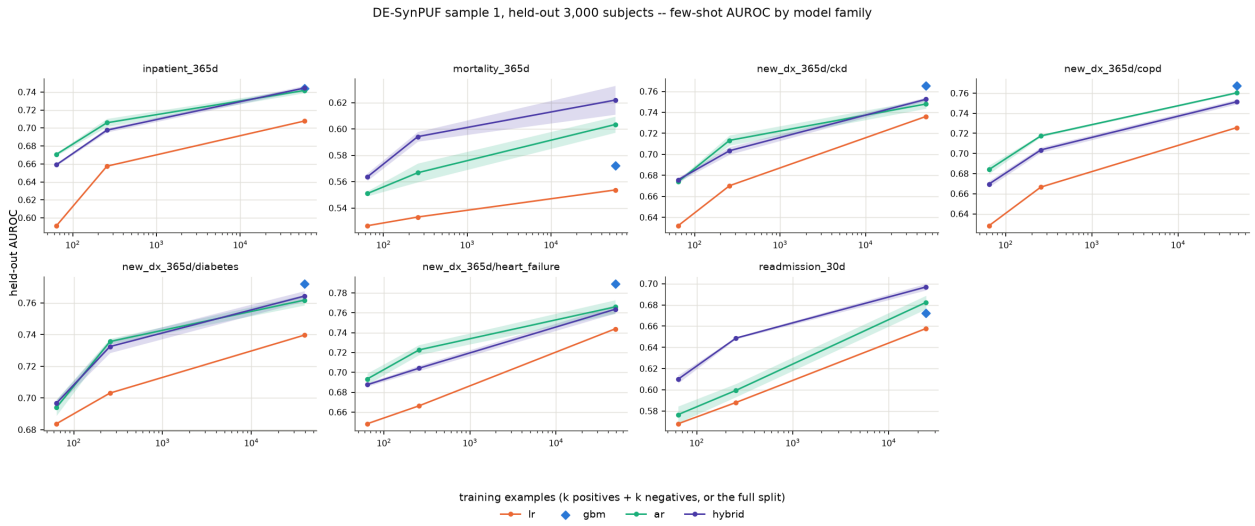

Figure 6: Held-out AUROC against few-shot training size, one panel per task, one line per model family.

Table 6: All trained cells across the three token budgets. The 48M rows are the pilot grids (4-layer, 192-wide, Apple M4/MPS); the 200M and 1B rows are the scale grids (6-layer, 256-wide, RTX 4060). Every row is seed 0 except the 1B `ar` and hybrid rows, which are the 3-seed mean (seeds 0, 1, 2; see Table 8 for per-seed values, std and overlap). `gbm` and `lr` do not depend on the encoder and are reused at every budget.

cell	scale	family	inpatient	mortality	ckd	copd	diabetes	heart fail.	readmiss.	mean	gain
hybrid	48M	nextlatent	0.7417	0.6332	0.7531	0.7487	0.7618	0.7642	0.6985	0.7287	+0.0923
`ar`	48M	ar	0.7265	0.6060	0.7586	0.7552	0.7590	0.7437	0.6946	0.7205	+0.0445
`gbm`	—	count	0.7440	0.5723	0.7654	0.7674	0.7721	0.7893	0.6724	0.7261	—
`lr`	—	count	0.7077	0.5537	0.7361	0.7258	0.7397	0.7438	0.6578	0.6949	—
hybrid	200M	nextlatent	0.7511	0.5919	0.7761	0.7676	0.7729	0.7767	0.6930	0.7328	+0.0781
`ar`	200M	ar	0.7515	0.5946	0.7624	0.7747	0.7714	0.7908	0.6674	0.7304	+0.0757
`recon_only`	200M	recon-only	0.7326	0.6508	0.7336	0.7242	0.7568	0.7453	0.6894	0.7190	+0.0448
`jepa_ema`	200M	masked-span	0.6948	0.6413	0.7108	0.7189	0.7499	0.7159	0.6580	0.6985	+0.0243
hybrid (3-seed)	1B	nextlatent	0.7533	0.5866	0.7737	0.7780	0.7783	0.7987	0.6853	**0.7363**	+0.0816
`ar` (3-seed)	1B	ar	0.7452	0.5997	0.7584	0.7674	0.7615	0.7796	0.6637	0.7251	+0.0704

0.7624 → 0.7584, `copd` 0.7747 → 0.7674, `diabetes` 0.7714 → 0.7615, `heart_failure` 0.7908 → 0.7796, and `readmission_30d` 0.6674 → 0.6637. The hybrid improves at every step: 0.7287 → 0.7328 → 0.7363 (1B a 3-seed mean), that is +0.0041 then +0.0035. At 1B (3-seed means) the hybrid leads `ar` on six of seven tasks — every task but `mortality_365d` — by 0.81 to 2.16 AUROC points (`inpatient` +0.81, `ckd` +1.53, `copd` +1.05, `diabetes` +1.68, `heart_failure` +1.92, `readmission` +2.16); `ar` leads on `mortality_365d` by 1.31 points. The hybrid at 1B (0.7363, 3-seed mean) is the highest mean AUROC of any cell in the table, above `gbm` at 0.7261 and `lr` at 0.6949. `recon_only` and `jepa_ema` were run through 200M only.

6.6 Seed replication at 1B

A seed-replication grid at the 1B budget (`scale1b-seeds-desynpuf`: seeds 1 and 2 for both `ar` and the hybrid, four cells) has completed, giving three seeds (0, 1, 2) per cell. Table 8 gives every per-seed value plus the per-task mean and sample standard deviation (n=3) for `ar` and the hybrid.

Per-task overlap. Comparing the min-max range across the three seeds, `ar` and the hybrid do not overlap on `ckd` (`ar` 0.7581–0.7586 vs. hybrid 0.7717–0.7752), `diabetes` (0.7589–0.7639 vs. 0.7779–

Table 7: `ar` against the hybrid across the three token budgets, mean AUROC and the `readmission_30d` column. 48M and 200M rows are seed 0; the 1B row is the 3-seed mean (Table 8).

tokens	`ar` mean	`ar` readmission	hybrid mean	hybrid readmission
48M	0.7205	0.6946	0.7287	0.6985
200M	0.7304	0.6674	0.7328	0.6930
1B (3-seed)	0.7251	0.6637	**0.7363**	**0.6853**

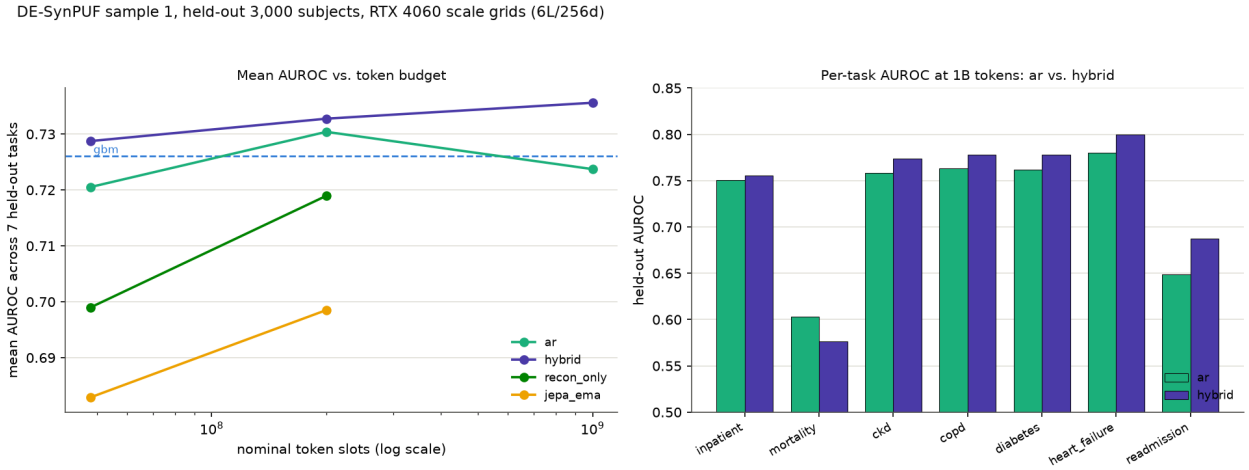

Figure 7: Left: mean AUROC against token budget (log x) for `ar`, the hybrid, `recon_only` and `jepa_ema`, with `gbm` as a horizontal reference; the 1B `ar`/hybrid points are 3-seed means with a min-max error bar. Right: per-task AUROC at 1B tokens, `ar` against the hybrid, paired bars, each a 3-seed mean with a min-max whisker.

0.7787), `heart_failure` (0.7787–0.7804 vs. 0.7950–0.8015) and `readmission_30d` (0.6490–0.6742 vs. 0.6840–0.6874). They overlap on `inpatient_365d` (0.7410–0.7506 vs. 0.7491–0.7556) and `mortality_365d` (0.5933–0.6031 vs. 0.5767–0.5969). `copd` does not overlap by the same min-max definition, but by the smallest margin of any non-overlapping task: `ar`'s maximum (0.7738) sits 0.0022 below the hybrid's minimum (0.7760).

The largest hybrid-over-`ar` margin at 1B, on `readmission_30d` (Table 6, +2.16 points on the 3-seed means), is also a task whose seed ranges do not overlap between the two objectives: every one of `ar`'s three seeds (0.6490, 0.6742, 0.6679) is below every one of the hybrid's three seeds (0.6874, 0.6846, 0.6840). `ar`'s within-family seed spread on this task (0.0131 std, the largest of any task for either objective) is therefore smaller than its gap to the hybrid, not comparable to it as the single-seed-1 replicate suggested.

6.7 Model size and hybrid ablations

A third grid (`ablate2-desynpuf`, 18 rows) asks two questions the 200M/1B grids above cannot: does the hybrid's lead over `ar` depend on model size, and which of the hybrid's own knobs (target mode, SIGReg weight, horizon set, reconstruction weight) matter? It trains `ar` and the hybrid at three sizes — small (4-layer, 192-wide, 4 heads), base (6-layer, 256-wide, the `scale` shape, re-scored from `runs/scale-desynpuf/{ar,hybrid}/final.pt` rather than retrained), and large (8-layer, 384-wide, 6 heads) — each at 200M nominal token slots, and sweeps four single-knob variants of the base-size hybrid. Every trained cell runs at two seeds (1 and 2); the re-scored base cells carry only their original seed 0. Unlike every grid above, evaluation here covers the full held-out split (11,708 subjects, not the seeded 3,000-subject subset), so these numbers are not directly comparable to Tables 3 and 6. Full protocol and per-task tables are in `docs/experiments/ABLATION_RESULTS.md`.

Table 8: 1B seed replication, complete: three seeds per cell for `ar` and the hybrid. `std` is the sample standard deviation across the three seeds (n=3). Seed 0 rows repeat Table 6's single-seed 1B values; the mean rows repeat Table 6's 3-seed 1B values.

cell	seed	inpatient	mortality	ckd	copd	diabetes	heart fail.	readmiss.	mean
`ar`	0	0.7506	0.6028	0.7586	0.7629	0.7618	0.7804	0.6490	0.7237
`ar`	1	0.7410	0.6031	0.7584	0.7738	0.7639	0.7796	0.6742	0.7277
`ar`	2	0.7441	0.5933	0.7581	0.7656	0.7589	0.7787	0.6679	0.7238
`ar`	mean	0.7452	0.5997	0.7584	0.7674	0.7615	0.7796	0.6637	**0.7251**
`ar`	std	0.0049	0.0056	0.0003	0.0057	0.0025	0.0009	0.0131	—
hybrid	0	0.7553	0.5767	0.7741	0.7781	0.7779	0.7997	0.6874	0.7356
hybrid	1	0.7491	0.5862	0.7717	0.7798	0.7787	0.8015	0.6846	0.7359
hybrid	2	0.7556	0.5969	0.7752	0.7760	0.7784	0.7950	0.6840	0.7373
hybrid	mean	0.7533	0.5866	0.7737	0.7780	0.7783	0.7987	0.6853	**0.7363**
hybrid	std	0.0037	0.0101	0.0018	0.0019	0.0004	0.0034	0.0018	—

Table 9: `ar` vs. hybrid at three sizes, full held-out split (11,708 subjects), 200M nominal token slots per cell. Small and large rows are 2-seed means; base rows are seed 0 only (re-scored `scale-desynpuf` checkpoints). Trainable parameter counts are computed from each model's actual construction (Appendix A's configs plus the grid's per-cell overrides), not estimated. *mean-6* excludes `mortality_365d`; *mean-7* includes it.

size	family	params	mean-6	mean-7
small	`ar`	7.70M	0.7460	0.7259
small	hybrid	8.01M	**0.7539**	**0.7364**
base (seed 0)	`ar`	12.7M	0.7507	0.7315
base (seed 0)	hybrid	13.2M	**0.7554**	**0.7345**
large	`ar`	26.2M	0.7540	0.7336
large	hybrid	27.4M	**0.7595**	**0.7382**

Size. The hybrid leads `ar` on the 6-task mean at every size: +0.0079 (small), +0.0047 (base), +0.0055 (large). The gap does not close as size increases — base's gap is the smallest of the three, and large's is larger than base's.

Knobs. No-SIGReg is the best knob at both seeds individually (0.7620 at seed 1, 0.7611 at seed 2), ahead of the other three variants at both seeds. Horizon-[1]-only (0.7512) and shared-target (0.7510) are the worst knobs, both below the 0.7554 default. `lambda_recon`=0.3 (0.7593) is within 0.0039 of the 0.1 default, inside the 0.0020–0.0046 seed spread measured among these four knobs.

Decision. Section 9 adopts a new default configuration (`configs/pretrain_default.yaml`) from this result: the hybrid objective, causal encoder, EMA target, horizons $[1, 4, 16]$, $\lambda_{\text{recon}} = 0.1$, and $\lambda_{\text{sig}} = 0$ — SIGReg dropped, per the no-SIGReg result above.

Caveats. The base-size default row is one seed, so every size/knob comparison against it is a 2-seed-vs-1-seed comparison, not 2-seed-vs-2-seed. 200M nominal token slots, DE-SynPUF only. Differences among the base-size hybrid knobs are 0.3–0.7 points (mean-6, vs. the default) against a seed spread of about 0.3–0.5 points among those same knobs.

Table 10: Base-size (6-layer, 256-wide) hybrid-knob comparison, mean AUROC across the six non-mortality tasks. *default* is `hybrid_base_s0` (seed 0 only); the other four rows are 2-seed means. *seed spread* is the per-task $|s1 - s2|$ averaged over the six tasks.

knob	mean-6	seed spread
default (`lambda_recon`=0.1, EMA target, horizons [1,4,16])	0.7554	—
shared target (instead of EMA)	0.7510	0.0029
no SIGReg (`lambda_sigreg`=0)	**0.7615**	0.0039
horizon [1] only (instead of [1,4,16])	0.7512	0.0020
`lambda_recon`=0.3 (instead of 0.1)	0.7593	0.0046

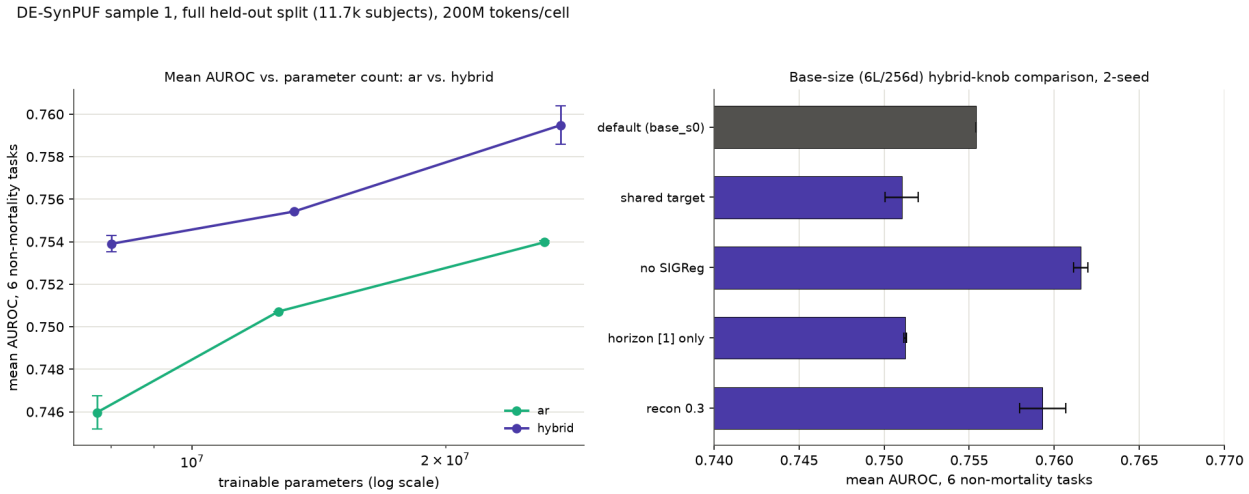

Figure 8: Left: mean AUROC across the six non-mortality tasks against trainable parameter count (log x) for `ar` and the hybrid across the three sizes, with 2-seed min-max error bars (none at base, seed 0 only). Right: the base-size hybrid-knob comparison as horizontal bars with 2-seed min-max whiskers (none for the default row). Regenerated by `scripts/plot_ablate2.py`, which builds each model from its config overrides to compute the parameter counts rather than reading a hand-typed estimate.

7 Discussion

7.1 What the evidence supports

Density and discreteness of supervision, not the latent target, moved the probe at these budgets. Three rows carry this. Masked-span latent JEPA, swept across target mode, SIGReg weight and masking mix, lands within $[-0.0046, +0.0080]$ of its own untrained control. Adding a discrete auxiliary term to the same architecture takes it to +0.0177 to +0.0296. Removing the latent term and keeping only the discrete one — `recon_only`, $\lambda_{\text{pred}} = 0$ — retains +0.0254, which is 0.0041 AUROC below the cell that keeps both. At this budget the latent prediction term contributes little that the auxiliary code term does not already contribute. Grid 4 was designed on the reading that two things separate AR from masked-span JEPA — a loss term at *every* position rather than at the 10–30% a mask covers, and a *discrete* target — and took the density while keeping the latent target. The dense latent cells (`nextlatent_h1`, `nextlatent_h1416`) gain +0.0305 and +0.0340 over their control, above every masked-span cell, while still sitting below AR in absolute terms.

The hybrid is above either of its components at all three budgets. `nextlatent_h1416_recon` scores 0.7287 at 48M against `ar`'s 0.7205 and `nextlatent_h1416`'s 0.6704, and it is the top row of all 20 pilot cells by both gain and absolute mean. At 200M it is 0.7328 against 0.7304; at 1B (3-seed means), 0.7363 against 0.7251. Because setting $\lambda_{\text{pred}} = 0$ reduces the hybrid to AR exactly, this is a controlled addition rather than a different model. The 48M seed replication puts the three-seed means at 0.7234 (`ar`) and 0.7279 (hybrid), a gap of 0.0045; the 1B seed replication

(Section 6.6) puts the gap at 0.0112, with the hybrid's per-task 3-seed range above `ar`'s on four tasks where the ranges do not overlap at all (`ckd`, `diabetes`, `heart_failure`, `readmission_30d`).

The AR plateau from 200M to 1B. `ar` gains 0.0099 from 48M to 200M and loses 0.0053 from 200M to 1B (3-seed mean), with six of seven tasks moving down over the second step — every task but `mortality_365d`. The hybrid gains at both steps, though by decreasing amounts (+0.0041, +0.0035). One confound remains: the 48M points were trained on different hardware with a different and smaller architecture, so the 48M-to-200M step is not a clean token-budget step; the 200M point is also still a single seed, so the 200M-to-1B step compares one run to a three-seed mean. The 1B point itself no longer carries a single-seed caveat: three seeds put `ar`'s 1B mean at 0.7251 with a per-task std of 0.0003 to 0.0131 (Table 8), and the drop from 200M persists at every one of the three seeds (0.7237, 0.7277, 0.7238, all below 200M's 0.7304).

The readmission signature. `readmission_30d` behaves differently from the other six tasks in three places. It is the task where the count-feature `gbm` baseline is weakest relative to the encoders (0.6724, below both 48M encoder cells). It is the task where `ar` degrades monotonically with token budget — 0.6946, 0.6674, 0.6637 (3-seed mean) — while the hybrid declines far less — 0.6985, 0.6930, 0.6853 (3-seed mean) — producing a 2.16-point gap at 1B, the largest of any task, and the only task at 1B whose three-seed `ar` and hybrid ranges do not overlap by any margin: `ar`'s worst seed (0.6742) is still below the hybrid's worst seed (0.6840). And it is the task with the largest few-shot separation: at k=128 the hybrid reads 0.6487 against `ar`'s 0.5994, and at 48M with three seeds the hybrid-over-`ar` gap (0.6969 vs 0.6824) exceeds both families' seed ranges. Readmission is also the only task anchored on a *specific event type* (an inpatient discharge) rather than on a hash-drawn time, and the only one with a 30-day rather than a 365-day horizon. A short horizon anchored on a discharge is the task most plausibly served by a representation that predicts what happens next in *time* rather than what codes are prevalent, which is what the dense next-latent term at horizons $\{1, 4, 16\}$ supplies. That is a hypothesis consistent with the numbers, not something these experiments establish. Note also that both families decline on this task as the budget grows, which nothing here explains.

7.2 What the evidence does not support

- **No claim that latent prediction cannot work for EHR.** What was measured is that five masked-span variants, at 48M tokens with a 7.9M-parameter model on a labs-free synthetic claims file, did not beat their untrained control on a frozen linear probe. The single masked-span cell taken to 200M (`jepa_ema`) does clear its control there, by +0.0243. The budget, the model size, the source and the readout are all confounded with the objective.
- **No claim of comparability to published EHR results.** No experiment in this paper touches MIMIC-IV or EHRSHOT. DE-SynPUF's construction weakens history-to-future association by design.
- **No significance test on the 1B lead, and only three seeds.** Table 8 gives three training runs per cell for `ar` and the hybrid at 1B, not a hypothesis test; no p-value or confidence interval over seeds is computed anywhere in this paper. The per-task min-max ranges do not overlap on four of seven tasks and do overlap on two (`inpatient_365d`, `mortality_365d`), which is evidence for a real per-task difference on those four, not proof of one, and three seeds is too few to characterize the tails of the across-seed distribution.
- **No claim about calibration or clinical utility.** AUROC is the metric reported throughout. AUPRC, Brier and calibration slope are computed and stored for every cell but are not analysed here.
- **No claim about grid-1 numbers against grid 2–5 numbers.** They were read off the same encoders through different pooling. Only within-grid, control-matched gains are comparable

across grids.

8 Limitations

1. **Single seed at 200M; three seeds at 1B for two of four objectives.** The 1B `ar` and hybrid cells have three seeds each (Section 6.6); every 200M cell (`ar`, hybrid, `recon_only`, `jepa_ema`) is seed 0 only, so the 200M-to-1B comparison in Table 6 rests on a single 200M run against a three-seed 1B mean. `recon_only` and `jepa_ema` have no seed-replication grid at any budget and have not been run at 1B. At 48M, only `ar` and the hybrid have three seeds; every masked-span, JEPA+recon, recon-only and window cell in Table 3 is seed 0 alone.

2. **Claims data without laboratory values.** DE-SynPUF has no lab results, no vitals and no notes. The value channel carries little (length of stay, days supplied), so the value quantizer and the value embedding path — a substantial part of the architecture — are barely exercised. Low absolute AUROCs are partly a property of the source.

3. **A 3,000-subject held-out subset,** not the full held-out split. The cut is seeded, task-independent and applied identically to every model, but bootstrap intervals are correspondingly wider than the full split would give, and the paper reports point estimates in its tables.

4. **The embedding table dominates the parameter count.** At the pilot scale it is 5,892,288 of 7,959,680 trainable parameters (74%), of which 5.76M is the $30{,}000 \times 192$ code table. At the scale config it is 7,905,536 of 13,208,336 for the hybrid (60%) and of 12,665,104 for `ar` (62%). Encoder capacity is a minority of what is being trained at both sizes, so "scaling tokens" here is not scaling a model in the usual sense.

5. **Small models, short budgets.** 4-layer/192-wide and 6-layer/256-wide encoders; 48M nominal token slots is about 2.0 epochs of the train split, and windows are 47% full at `max_len` 256, so nominal slots overstate real events by roughly a factor of two. None of these budgets is a statement about what an objective can do with more.

6. **Hardware and architecture change together across budgets.** The 48M points are Apple M4/MPS at 4×192 in fp32; the 200M and 1B points are RTX 4060 at 6×256 in bf16. The 48M-to-200M step is therefore not a clean token-budget comparison.

7. **Frozen linear probes are the only readout.** No fine-tuning was run. An objective that produces a representation a linear probe cannot read but a fine-tune can would look like a null result here.

8. **No MIMIC-IV or EHRSHOT results.** The ETL for MIMIC-IV exists and is tested against the demo subset, but no pretraining or evaluation run on it is reported.

9. **The diagnostics in Section 6.2 are not reproducible from the repository.** They were one-off measurements; their numbers and definitions are recorded in the protocol documents, but no committed script regenerates them. One of the four did not survive re-measurement.

9 Roadmap

In the order they are queued. `configs/pretrain_default.yaml` (Section 6.7) — the hybrid objective, causal encoder, EMA target, horizons $[1, 4, 16]$, $\lambda_{\text{recon}} = 0.1$, $\lambda_{\text{sig}} = 0$ (SIGReg dropped) — is now the default for every item below that trains a model, unless the item says otherwise.

1. Reseed the 200M grid (`ar`, hybrid, `recon_only`, `jepa_ema`) so the 200M-to-1B step is a matched-seed-count comparison, now that 1B has three seeds for `ar` and the hybrid.

2. Take `recon_only` and a masked-span cell to 1B, so that the objective-family ordering at 48M and 200M can be checked at the largest budget rather than inferred.

3. Confirm the no-SIGReg default at the 200M/1B scale and, if it holds, reseed the 1B hybrid cells at $\lambda_{\text{sig}} = 0$ so Table 6's scaling result is reported under the default configuration rather than the $\lambda_{\text{sig}} = 0.05$ setting it currently uses.
4. Pretrain and evaluate on the full MIMIC-IV v3.1 extract, where laboratory values exist and the value path is actually exercised.
5. Evaluate against the EHRSHOT task suite and MEDS-DEV, so that numbers become comparable to published work rather than only to internal controls.
6. Add a fine-tuning readout alongside the frozen probe, to separate "the representation does not contain it" from "a linear probe cannot read it".
7. Release pretrained weights and a citable version of this document.

Acknowledgments

This work uses the CMS 2008–2010 Data Entrepreneurs' Synthetic Public Use File (DE-SynPUF), released by the Centers for Medicare & Medicaid Services; the MIMIC-IV demo subset, distributed by PhysioNet under the Open Data Commons Open Database License; and records generated by Synthea. Development of the data path used all three; every result reported here is on DE-SynPUF alone. The design draws on the JEPA line of work — I-JEPA, V-JEPA 2 and LeJEPA — for the latent-prediction formulation and the SIGReg anti-collapse objective, and on MEDS and MEDS-DEV/ACES for the data and task standards. No code from those projects is included in this implementation.

References

[1] Irsyad Adam, Zekai Chen, David Laprade, Shaun Porwal, David Laub, Erik Reinertsen, Arda Pekis, and Kevin Brown. The patient is not a moving document: A world model training paradigm for longitudinal EHR. *arXiv preprint arXiv:2601.22128*, 2026. URL `https://arxiv.org/abs/2601.22128`. Introduces SMB-Structure.

[2] Bert Arnrich, Edward Choi, Jason Fries, Matthew McDermott, Jungwoo Oh, Tom Pollard, Nigam Shah, Ethan Steinberg, Michael Wornow, and Robin van de Water. Medical event data standard (MEDS): Facilitating machine learning for health. In *ICLR 2024 Workshop on Learning from Time Series for Health*, 2024. URL `https://openreview.net/forum?id=IsHy2ebjIG`.

[3] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. *arXiv preprint arXiv:2301.08243*, 2023. URL `https://arxiv.org/abs/2301.08243`.

[4] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL `https://arxiv.org/abs/2506.09985`.

[5] Randall Balestriero and Yann LeCun. LeJEPA: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025. URL `https://arxiv.org/abs/2511.08544`.

[6] Alexander Chemeris, Ming Jin, and Randall Balestriero. LeNEPA: No-augmentation next-latent prediction for time-series representation learning. *arXiv preprint arXiv:2607.00958*, 2026. URL `https://arxiv.org/abs/2607.00958`. KDD 2026 MILETS workshop.

[7] Sofiane Ennadir, Siavash Golkar, and Leopoldo Sarra. Joint embeddings go temporal. *arXiv preprint arXiv:2509.25449*, 2025. URL https://arxiv.org/abs/2509.25449.

[8] Adibvafa Fallahpour, Mahshid Alinoori, Wenqian Ye, Xu Cao, Arash Afkanpour, and Amrit Krishnan. EHRMamba: Towards generalizable and scalable foundation models for electronic health records. *arXiv preprint arXiv:2405.14567*, 2024. URL https://arxiv.org/abs/2405.14567.

[9] Lin Lawrence Guo, Santiago Eduardo Arciniegas, Joseph Jihyung Lee, Adam Paul Yan, George Tomlinson, Jason Fries, and Lillian Sung. Tokenization tradeoffs in structured EHR foundation models. *arXiv preprint arXiv:2603.15644*, 2026. URL https://arxiv.org/abs/2603.15644.

[10] Vincent Jeanselme, Zilin Jing, Aparajita Kashyap, Chao Pang, Florent Pollet, Young Sang Choi, and Shalmali Joshi. FoMoH: A clinically meaningful foundation model evaluation for structured electronic health records. *arXiv preprint arXiv:2505.16941*, 2025. URL https://arxiv.org/abs/2505.16941.

[11] Xianhang Li, Chen Huang, Chun-Liang Li, Eran Malach, Josh Susskind, Vimal Thilak, and Etai Littwin. Rethinking JEPA: Compute-efficient video SSL with frozen teachers. *arXiv preprint arXiv:2509.24317*, 2025. URL https://arxiv.org/abs/2509.24317.

[12] Nassim Oufattole, Teya Bergamaschi, Aleksia Kolo, Hyewon Jeong, Hanna Gaggin, Collin M. Stultz, and Matthew B. A. McDermott. MEDS-Tab: Automated tabularization and baseline methods for MEDS datasets. *arXiv preprint arXiv:2411.00200*, 2024. URL https://arxiv.org/abs/2411.00200.

[13] Chao Pang, Xinzhuo Jiang, Nishanth Parameshwar Pavinkurve, Krishna S. Kalluri, Elise L. Minto, Jason Patterson, Linying Zhang, George Hripcsak, Gamze Gürsoy, Noémie Elhadad, and Karthik Natarajan. CEHR-GPT: Generating electronic health records with chronological patient timelines. *arXiv preprint arXiv:2402.04400*, 2024. URL https://arxiv.org/abs/2402.04400.

[14] Pawel Renc, Yugang Jia, Anthony E. Samir, Jaroslaw Was, Quanzheng Li, David W. Bates, and Arkadiusz Sitek. Zero shot health trajectory prediction using transformer. *arXiv preprint arXiv:2407.21124*, 2024. URL https://arxiv.org/abs/2407.21124. Introduces ETHOS.

[15] Artem Shmatko, Alexander Wolfgang Jung, Kumar Gaurav, Søren Brunak, Laust Hvas Mortensen, Ewan Birney, Tom Fitzgerald, and Moritz Gerstung. Learning the natural history of human disease with generative transformers. *Nature*, 647(8088):248–256, 2025. doi: 10.1038/s41586-025-09529-3. Introduces Delphi-2M.

[16] Ethan Steinberg, Jason Fries, Yizhe Xu, and Nigam Shah. MOTOR: A time-to-event foundation model for structured medical records. *arXiv preprint arXiv:2301.03150*, 2023. URL https://arxiv.org/abs/2301.03150.

[17] Michael Wornow, Rahul Thapa, Ethan Steinberg, Jason A. Fries, and Nigam H. Shah. EHRSHOT: An EHR benchmark for few-shot evaluation of foundation models. *arXiv preprint arXiv:2307.02028*, 2023. URL https://arxiv.org/abs/2307.02028.

[18] Michael Wornow, Suhana Bedi, Miguel Angel Fuentes Hernandez, Ethan Steinberg, Jason Alan Fries, Christopher Ré, Sanmi Koyejo, and Nigam H. Shah. Context clues: Evaluating long context models for clinical prediction tasks on EHRs. *arXiv preprint arXiv:2412.16178*, 2024. URL https://arxiv.org/abs/2412.16178.

[19] Justin Xu, Jack Gallifant, Alistair E. W. Johnson, and Matthew B. A. McDermott. ACES: Automatic cohort extraction system for event-stream datasets. *arXiv preprint arXiv:2406.19653*, 2024. URL https://arxiv.org/abs/2406.19653. ICLR 2025.

[20] Yixuan Yang, Mehak Arora, Ryan Zhang, Baraa Abed, Junseob Kim, Tilendra Choudhary, and Rishikesan Kamaleswaran. Clin-JEPA: A multi-phase co-training framework for joint-embedding predictive pretraining on EHR patient trajectories. *arXiv preprint arXiv:2605.10840*, 2026. URL `https://arxiv.org/abs/2605.10840`.

[21] Sheng Zhang, Qin Liu, Naoto Usuyama, Cliff Wong, Tristan Naumann, and Hoifung Poon. Exploring scaling laws for EHR foundation models. *arXiv preprint arXiv:2505.22964*, 2025. URL `https://arxiv.org/abs/2505.22964`.

A Reproducibility

A.1 Configurations

Table 11: The two base configurations. Every cell in a grid is these values plus the one-line override the grid file specifies.

	`configs/pretrain_pilot.yaml`	`configs/pretrain_scale.yaml`
encoder	4 layers, 192 wide, 4 heads	6 layers, 256 wide, 4 heads
predictor	2 layers, 96 wide, 4 heads	4 layers, 128 wide, 4 heads
feed-forward	SwiGLU, ratio 4.0 (hidden 512)	SwiGLU, ratio 4.0 (hidden 680)
dropout / attn dropout	0.1 / 0.0	0.1 / 0.0
position	rotary, base 10^4	rotary, base 10^4
Fourier frequencies	16	16
`max_len` / `min_len`	256 / 16	512 / 16
batch	64	64
window sampling	`random_window`	`random_window`
target mode / EMA	`ema`, 0.996 $\rightarrow$ 1.0 linear	`ema`, 0.996 $\rightarrow$ 1.0 linear
λ_{pred}	1.0	1.0
λ_{sig}	0.05	0.05
λ_{recon} (where used)	0.1	0.1
smooth-L_1 β	1.0	1.0
SIGReg directions / grid / $t_{\max}$ / σ	256 / 17 / 5.0 / 1.0	256 / 17 / 5.0 / 1.0
SIGReg max rows	8192	8192
p_{future}	0.6	0.6
optimizer	AdamW, $\beta = (0.9, 0.95)$	AdamW, $\beta = (0.9, 0.95)$
learning rate	3.0e-4	3.0e-4
schedule	cosine to 1%, 100-step linear warmup	same
weight decay	0.05 (matrices only)	0.05 (matrices only)
gradient clip	1.0	1.0
precision	float32 (`auto` on MPS)	bf16
budget	48,005,120 slots, 2,930 steps	200M (6,104 steps) / 1B (30,518 steps)
vocabulary	30,000	30,000
hardware	Apple M4, 16 GB, MPS	RTX 4060, 8 GB, CUDA
throughput	4.8k–9.5k tok/s	$\approx$33k tok/s

A.2 Commands

Build the MEDS extract and the tensor cache:

```
python -m ehrjepa.data.etl desynpuf --input <raw> --output data/meds/desynpuf-s1
python -m ehrjepa.data.tokenize build data/meds/desynpuf-s1 \
    --cache data/cache/desynpuf-s1 --ndc-digits 9 --max-vocab 30000
```

Build the task labels (once; reused by every grid):

```
python -m ehrjepa.eval.tasks --source desynpuf-s1
```

Run an ablation grid (resumable at cell granularity; a cell already present in `summary.json` is skipped):

```
python scripts/ablate.py configs/grids/pilot4_desynpuf.yaml --dry-run
nohup python scripts/ablate.py configs/grids/pilot4_desynpuf.yaml &
tail -f runs/2026-09-04-pilot4-desynpuf/ablate.log
```

Evaluate checkpoints directly:

```
python -m ehrjepa.eval.run --source desynpuf-s1 --tasks all \
    --models lr,gbm,random_init,ckpt:runs/<grid>/<cell>/final.pt \
    --out docs/experiments/<dir>/ \
    --bootstrap 200 --eval-subject-limit 3000 --eval-subject-seed 0
```

Regenerate the figures from the committed result tables:

```
python scripts/plot_grids.py      # figures/pilot_grids_gain.png
python scripts/plot_pilot.py      # figures/pilot_desynpuf_auroc.png
python scripts/plot_fewshot.py    # figures/fewshot_desynpuf.png
python scripts/plot_scale.py      # figures/scale_desynpuf.png
```

A.3 Commit hashes for the key result commits

Table 12: The commit at which each result in this paper was recorded. Each grid's directory under `docs/experiments/` contains a `README.md` written *before* its numbers existed (the protocol) and a `summary.md` appended by the runner as each cell finished (the numbers).

commit	date	result
`9308019`	2026-09-03	the three MPS pretraining sanity runs
`1bf4bc4`	2026-09-03	pilot grid 1: six objectives at 48M tokens with controls
`b7aca44`	2026-09-04	pilot grid 2: content targets, mask-token time, code reconstruction
`1c43622`	2026-09-04	pilot grid 3: code reconstruction with and without the latent term
`55f90d0`	2026-09-04	pilot grid 4: dense next-latent and future-window JEPA
`d64d7ab`	2026-09-04	pilot grid 5: seed replication of `ar` and the hybrid
`f0492c0`	2026-09-04	few-shot evaluation for `lr`/`gbm`/`ar`/hybrid
`daece7c`	2026-09-04	`PILOT_RESULTS.md`, grids 1–4 consolidated
`1d8998c`	2026-09-06	200M-token scale grid on the RTX 4060
`8721a62`	2026-09-06	fix: checkpoint eval cache keyed by content fingerprint
`09f4405`	2026-09-06	1B-token grid, and the seed-replication grid file
`95c8e35`	2026-09-06	fix: the 1B seed grid, four cells at seeds 1 and 2
`ce966b2`	2026-09-06	`SCALE_RESULTS.md`, the 200M/1B grids consolidated

Weight decay is applied to matrices only: every parameter with fewer than two dimensions, every name containing `emb`, the `[CLS]` token and the predictor's `[MASK]` token are excluded. Step counts are derived as $\lceil \text{budget}/(\text{batch} \times \text{max\_len}) \rceil$.

One inconsistency to be aware of when reproducing. The header comment in `configs/grids/scale_desynpuf.yaml` describes the grid as "four cells at 500M nominal token slots each", but the machine-readable field in the same file reads `budget_tokens: 200000000`. 200M is what the runner uses and what every results document reports; the comment is stale.

A.4 Determinism

Three seeded draws control what is compared, and all three are pure functions of the subject identifier via `blake2b`, so none depends on row order, shard count or library version: the 80/10/10

split (seed 20240917, pinned by a test), the anchor draw (seed 20260903), and the 3,000-subject evaluation cut (seed 0). Per-subject held-out scores for every (task, model) pair are written to `predictions.parquet`, so a metric can be recomputed without refitting anything, and `python -m ehrjepa.eval.report <results.json>` re-renders a report without refitting.